

**PENGEMBANGAN DASHBOARD ESTIMASI BEBAN TUGAS
MAHASISWA BERBASIS RETRIEVAL-AUGMENTED
GENERATION (RAG)
(STUDI KASUS: TUGAS AKADEMIK SISTEM INFORMASI)**



**MUHAMMAD ISWARI
H071221007**



**PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

**PENGEMBANGAN DASHBOARD ESTIMASI BEBAN TUGAS
MAHASISWA BERBASIS *RETRIEVAL-AUGMENTED*
GENERATION (RAG)
(STUDI KASUS: TUGAS AKADEMIK SISTEM INFORMASI)**

**MUHAMMAD ISWARI
H071221007**



**PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

**PENGEMBANGAN DASHBOARD ESTIMASI BEBAN TUGAS
MAHASISWA BERBASIS *RETRIEVAL-AUGMENTED
GENERATION* (RAG)
(STUDI KASUS: TUGAS AKADEMIK SISTEM INFORMASI)**

MUHAMMAD ISWARI
H071221007

Skripsi

sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Sistem Informasi

Pada

**PROGRAM STUDI SISTEM INFORMASI
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS HASANUDDIN
2026**

SKRIPSI

**PENGEMBANGAN DASHBOARD ESTIMASI BEBAN TUGAS
MAHASISWA BERBASIS *RETRIEVAL-AUGMENTED*
GENERATION (RAG)
(STUDI KASUS : TUGAS AKADEMIK SISTEM INFORMASI)**

MUHAMMAD ISWARI
H071221007

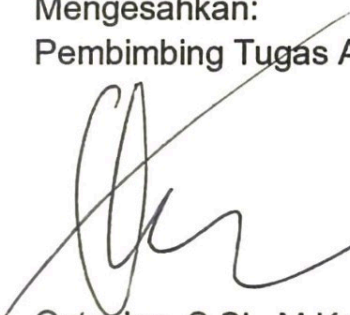
Skripsi,

Telah dipertahankan di depan Panitia Ujian Sarjana Sistem Informasi pada tanggal 14 Agustus 2026 dan dinyatakan telah memenuhi syarat kelulusan pada

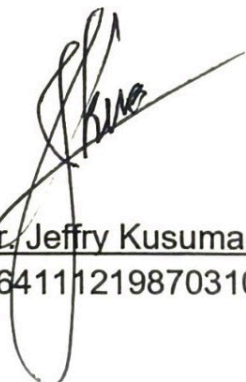
Program Studi Sistem Informasi
Departemen Matematika
Fakultas Matematika dan Ilmu Pengetahuan Alam
Universitas Hasanuddin
Makassar

Mengesahkan:
Pembimbing Tugas Akhir,

Mengetahui:
Ketua Program Studi,


Octavian, S.Si., M.Kom
NIP 199210142024123001




Prof. Dr. Jeffry Kusuma, Ph.D.
NIP 196411121987031002

PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa skripsi yang berjudul "Pengembangan Dashboard Estimasi Beban Tugas Mahasiswa Berbasis Retrieval-Augmented Generation (RAG) (Studi Kasus

: Tugas Akademik Sistem Informasi)" adalah benar karya saya dengan arahan dari pembimbing (Octavian, S.Si., M.Kom). Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tulis saya berupa skripsi ini kepada Universitas Hasanuddin.

Makassar, 14 Agustus 2026



Muhammad Iswari
NIM H071221007

UCAPAN TERIMA KASIH

Puji syukur penulis panjatkan ke hadirat Allah Swt. atas segala rahmat, karunia, hidayah, serta pertolongan-Nya yang senantiasa dilimpahkan kepada penulis. Berkat izin dan ridha-Nya, penulis diberikan kesehatan, kekuatan, kesabaran, dan kemudahan dalam menghadapi berbagai tantangan selama proses penelitian hingga akhirnya dapat menyelesaikan penyusunan skripsi ini dengan baik. Semoga karya sederhana ini dapat memberikan manfaat bagi pengembangan ilmu pengetahuan dan bagi para pembaca.

Dengan penuh rasa hormat dan penghargaan, penulis menyampaikan terima kasih yang sebesar-besarnya kepada Bapak Octavian, S.Si., M.Kom., selaku dosen pembimbing yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan, arahan, motivasi, serta saran yang sangat berharga selama proses penyusunan skripsi ini. Penulis juga menyampaikan terima kasih kepada Bapak Prof Dr.Eng. Armin Lawi, S.Si., M.Eng. dan Bapak Edy Saputra Rusdi, S.Si., M.Si. selaku dosen penguji atas segala masukan, kritik, dan saran yang membangun demi penyempurnaan skripsi ini.

Rasa hormat dan terima kasih yang tak terhingga penulis persembahkan kepada kedua orang tua tercinta atas doa, kasih sayang, pengorbanan, serta dukungan moril dan materiel yang tiada henti. Ucapan terima kasih juga penulis sampaikan kepada seluruh keluarga atas semangat dan perhatian yang senantiasa diberikan.

Penulis menyampaikan apresiasi dan terima kasih yang tulus kepada seluruh mahasiswa yang berpartisipasi sebagai evaluator dalam penelitian ini atas waktu, perhatian, serta masukan yang diberikan sehingga proses evaluasi dan penyempurnaan hasil penelitian dapat terlaksana dengan baik.

Selanjutnya, penulis mengucapkan terima kasih kepada pimpinan dan seluruh sivitas akademika Universitas Hasanuddin atas kesempatan dan fasilitas yang diberikan selama menempuh pendidikan. Ucapan terima kasih juga penulis sampaikan kepada teman-teman Multi Gacore, teman-teman magang BBPP Batangkaluku tahun 2025, rekan-rekan KKN-T 114 Desa Darubiah, serta seluruh teman angkatan Sistem Informasi 2022 atas kebersamaan, dukungan, dan semangat yang diberikan selama ini.

Akhir kata, penulis mengucapkan terima kasih yang sebesar-besarnya kepada seluruh pihak yang telah disebutkan maupun yang tidak dapat disebutkan satu per satu atas segala bantuan, dukungan, doa, dan kerja sama yang diberikan selama proses penyusunan skripsi ini. Semoga segala kebaikan dan ketulusan yang telah diberikan mendapatkan balasan yang berlipat ganda serta keberkahan dari Allah Swt.

ABSTRAK

MUHAMMAD ISWARI. **Pengembangan Dashboard Estimasi Beban Tugas Mahasiswa Berbasis Retrieval-Augmented Generation (RAG) (Studi Kasus: Tugas Akademik Sistem Informasi)** (dibimbing oleh Octavian, S.Si., M.Kom).

Latar Belakang. Perkembangan kecerdasan buatan (Artificial Intelligence) membuka peluang pemanfaatan data akademik untuk mendukung proses pembelajaran di perguruan tinggi. Namun, pemanfaatan data historis tugas untuk menghasilkan estimasi beban tugas akademik masih relatif terbatas. Oleh karena itu, diperlukan pendekatan yang mampu menghasilkan estimasi beban tugas yang kontekstual dan dapat dijelaskan. **Tujuan.** Penelitian ini bertujuan untuk mengimplementasikan dan mengevaluasi dashboard estimasi beban tugas mahasiswa berbasis Retrieval-Augmented Generation (RAG), serta menilai efektivitas pendekatan RAG dalam melakukan estimasi beban tugas akademik. **Metode.** Penelitian menggunakan pendekatan kuantitatif dengan metode eksperimen melalui pengembangan dashboard berbasis Streamlit. Dataset terdiri atas 278 data tugas akademik yang berasal dari riwayat tugas Program Studi Sistem Informasi dan data sintetis yang telah divalidasi. Sistem memanfaatkan model multilingual-e5-base, FAISS, dan LLaMa 3.1 8B dalam arsitektur RAG. Evaluasi dilakukan menggunakan Hit Rate, Precision Rate, manual judgement, LLM-as-a-Judge, dan Parsing Success Rate (PSR). **Hasil.** Sistem memperoleh Hit Rate@3 sebesar 100% dan Precision@3 sebesar 80%. Evaluasi manual judgement menghasilkan skor rata-rata 4,16 untuk kesesuaian deadline, 3,94 untuk realisme estimasi waktu pengerjaan, dan 3,98 untuk kesesuaian tingkat kesulitan. Evaluasi LLM-as-a-Judge memperoleh skor rata-rata 4,07, sedangkan Parsing Success Rate mencapai 100%. **Kesimpulan.** Pendekatan Retrieval-Augmented Generation terbukti dapat menghasilkan estimasi beban tugas akademik berbasis referensi historis. Sistem mampu menghasilkan estimasi yang dinilai realistis, menyediakan reasoning yang relevan, serta menghasilkan keluaran yang konsisten dan terstruktur melalui dashboard yang dikembangkan.

Kata Kunci: *Retrieval-Augmented Generation*, Estimasi Beban Tugas Akademik, FAISS, LLaMa 3.1 8B, Streamlit.

ABSTRACT

MUHAMMAD ISWARI. **Development of a Retrieval-Augmented Generation (RAG)-Based Student Academic Workload Estimation Dashboard (Case Study: Information Systems Academic Assignments)** (supervised by Octavian, S.Si., M.Kom).

Background. The advancement of Artificial Intelligence (AI) has created opportunities to utilize academic data to support learning processes in higher education. However, the use of historical assignment data to estimate academic workload remains limited. Therefore, an approach capable of generating contextual and explainable workload estimations is needed. **Objective.** This study aims to implement and evaluate a Retrieval-Augmented Generation (RAG)-based student workload estimation dashboard and assess the effectiveness of the RAG approach in estimating academic workload. **Methods.** This study employed a quantitative approach with an experimental method through the development of a Streamlit-based dashboard. The dataset consisted of 278 academic assignment records collected from the Information Systems Study Program and validated synthetic data. The system utilized the multilingual-e5-base embedding model, FAISS, and LLaMa 3.1 8B within a RAG architecture. Evaluation was conducted using Hit Rate, Precision Rate, manual judgement, LLM-as-a-Judge, and Parsing Success Rate (PSR). **Results.** The system achieved a Hit Rate@3 of 100% and a Precision@3 of 80%. Manual judgement evaluation produced average scores of 4.16 for deadline suitability, 3.94 for the realism of estimated completion time, and 3.98 for difficulty suitability. The LLM-as-a-Judge evaluation obtained an average score of 4.07, while the Parsing Success Rate reached 100%. **Conclusion.** The Retrieval-Augmented Generation approach has been shown to produce academic task workload estimates based on historical references. The system is capable of generating estimates that are considered realistic, providing relevant reasoning, and producing consistent and structured outputs through the developed dashboard.

Keywords: Retrieval-Augmented Generation, Academic Workload Estimation, FAISS, LLaMa 3.1 8B, Streamlit.

DAFTAR ISI

	halaman
HALAMAN JUDUL	ii
HALAMAN PENGANTAR.....	iii
HALAMAN PENGESAHAN.....	iv
PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA.....	v
UCAPAN TERIMA KASIH.....	vi
ABSTRAK	vii
ABSTRACT	viii
DAFTAR ISI	ix
DAFTAR TABEL	xii
DAFTAR GAMBAR	xiii
DAFTAR LAMPIRAN	xiv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	3
1.3 Batasan Masalah	3
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	4
1.6 Landasan Teori	5
1.6.1 Sistem Informasi.....	5
1.6.2 Estimasi Beban Tugas Akademik (Academic Workload Estimation) ..	6
1.6.3 Kecerdasan Buatan (Artificial Intelligence)	7
1.6.4 Pemrosesan Bahasa Alami (Natural Language Processing).....	8
1.6.5 Representasi Teks dan Embedding	8
1.6.6 Semantic Similarity dan Cosine Similarity	10
1.6.7 Vector Database dan FAISS	11
1.6.8 Retrieval-Augmented Generation (RAG).....	11
1.6.9 Large Language Model (LLM)	13
1.6.10 Prompt Engineering.....	14
1.6.11 Data Sintetis dalam Pengembangan Sistem AI.....	14

1.6.12	Evaluasi Sistem RAG	15
1.6.13	Dashboard sebagai Media Visualisasi Data	18
1.6.14	Streamlit sebagai Framework Dashboard	19
1.6.15	LLaMa 3.1 8B.....	19
BAB II METODE PENELITIAN.....		21
2.1	Pendekatan Penelitian	21
2.2	Tempat dan Waktu Penelitian	21
2.3	Instrumen Penelitian	22
2.3.1	Perangkat Keras (Hardware).....	22
2.3.2	Perangkat Lunak (Software).....	22
2.4	Studi Kasus Penelitian	24
2.5	Dataset Penelitian	24
2.5.1	Sumber Data	24
2.5.2	Pengumpulan Data.....	26
2.5.3	Deskripsi Dataset	26
2.5.4	Proses Validasi dan Pembersihan Data	28
2.6	Perancangan Sistem.....	29
2.6.1	Pipeline Sistem.....	29
2.6.2	Indexing.....	30
2.6.3	Retrieval	33
2.6.4	Generasi Estimasi	36
2.6.5	Dashboard Streamlit.....	42
2.7	Metode Evaluasi.....	43
2.7.1	Evaluasi Hasil Retrieval (Hit Rate dan Precision Rate)	43
2.7.2	Evaluasi Kelayakan Hasil Estimasi.....	45
2.7.3	Evaluasi Kualitas Reasoning Menggunakan LLM-as-a-Judge	46
2.7.4	Evaluasi Parsing Success Rate (PSR).....	47
BAB III HASIL DAN PEMBAHASAN		49
3.....		49
3.1	Implementasi Sistem.....	49
3.2	Evaluasi Sistem.....	54
3.2.1	Evaluasi Retrieval.....	54

3.2.2	Evaluasi Manual Judgement	55
3.2.3	Evaluasi LLM-as-a-Judge.....	57
3.2.4	Evaluasi Parsing Success Rate.....	58
BAB IV KESIMPULAN DAN SARAN		60
4.1	Kesimpulan	60
4.2	Saran.....	61
DAFTAR PUSTAKA.....		62
LAMPIRAN		65

DAFTAR TABEL

Nomor Urut	Halaman
Tabel 1. Interpretasi Skala Likert 1–5.....	17
Tabel 2. Waktu Pelaksanaan Kegiatan.....	21
Tabel 3. Spesifikasi Perangkat Keras.....	22
Tabel 4. Daftar perangkat lunak dan pustaka yang digunakan dalam pengembangan sistem.....	22
Tabel 5 Distribusi jumlah data setiap mata kuliah pada dataset.....	26
Tabel 6. Struktur dan Atribut Dataset	28
Tabel 7. Struktur prompt sistem.....	38
Tabel 8. Struktur Keluaran Sistem.....	41
Tabel 9. Hasil Evaluasi Retrieval	54
Tabel 10. Hasil Evaluasi Manual Judgement Setiap Tugas.....	56
Tabel 11. Rata-Rata Setiap Aspek Evaluasi.....	56
Tabel 12. Hasil Evaluasi LLM-as-a-Judge	57

DAFTAR GAMBAR

Nomor Urut	Halaman
Gambar 1. Model Umum Sistem Informasi	5
Gambar 2. Ilustrasi proses embedding model	9
Gambar 3. Alur Proses RAG	13
Gambar 4. Pipeline Sistem RAG Estimasi Beban Tugas Akademik.....	29
Gambar 5. Tahapan dalam proses Indexing	31
Gambar 6. Alur Pemrosesan Inputan	33
Gambar 7. Tampilan awal sistem	50
Gambar 8. Tampilan hasil estimasi sistem	51
Gambar 9. Tampilan hasil estimasi sistem (2).....	53

DAFTAR LAMPIRAN

Nomor Urut	Halaman
Lampiran 1. Dataset Tugas Akademik.....	65
Lampiran 2. Preprocessing Dataset.....	65
Lampiran 3. Source Code Sistem	66
Lampiran 4. Evaluasi Retrieval	66
Lampiran 5. Evaluasi LLM-as-a-Judge	67
Lampiran 6. Evaluasi Manual Judgement.....	67

BAB I PENDAHULUAN

1.1 Latar Belakang

Pengelolaan pendidikan tinggi saat ini telah banyak didukung oleh teknologi informasi. Tidak hanya terbatas pada aspek administratif, berbagai perguruan tinggi mulai memanfaatkan data secara lebih strategis untuk mengevaluasi kualitas pembelajaran dan memahami beban akademik mahasiswa. Salah satu teknologi yang semakin berkembang dalam bidang pendidikan adalah kecerdasan buatan (Artificial Intelligence atau AI), yang memungkinkan analisis data akademik dilakukan secara otomatis dan berbasis data. Pemanfaatan AI dalam pendidikan mencakup berbagai aspek, seperti analitik pembelajaran, prediksi performa akademik, rekomendasi pembelajaran, serta analisis karakteristik tugas akademik untuk mendukung proses pembelajaran yang lebih efektif (Zawacki-Richter et al., 2019).

Di sisi lain, jenis dan bentuk tugas yang harus diselesaikan mahasiswa sekarang ini justru semakin beragam dan tidak sederhana. Mulai dari tugas individu yang menuntut pemahaman mendalam, sampai proyek kelompok yang memerlukan koordinasi dan pembagian peran, semuanya menguji kemampuan analisis serta penerapan konsep secara nyata. Akibat dari meningkatnya keragaman dan kompleksitas tugas ini, cara mahasiswa belajar ikut terpengaruh. Sebagian mahasiswa mulai kesulitan mengatur waktu, ada yang kehilangan strategi belajar yang efektif, dan tidak sedikit yang keterlibatannya dalam perkuliahan menurun (Pascoe et al., 2020). Lebih jauh lagi, jika beban akademik terasa terlalu berat, mahasiswa cenderung belajar hanya sekadar untuk lulus atau sekadar menyelesaikan tugas, sehingga kurang melibatkan daya pikir kritisnya. Padahal, proses berpikir mendalam justru menjadi inti dari pembelajaran di perguruan tinggi.

Kondisi ini terasa lebih menantang lagi pada program-program studi yang bersifat aplikatif, misalnya Program Studi Sistem Informasi. Pada program studi yang bersifat aplikatif, tugas akademik tidak hanya mengukur pemahaman konseptual, tetapi juga kemampuan merancang solusi, menganalisis sistem, dan melakukan implementasi teknis. Hal ini sejalan dengan karakteristik kurikulum Sistem Informasi yang menekankan pada pemecahan masalah nyata melalui pendekatan berbasis proyek dan integrasi antara konsep dan praktik (AIS & ACM, 2010). Selain itu, kompleksitas tugas yang tinggi dapat meningkatkan beban belajar mahasiswa, terutama apabila tugas tidak dirancang secara proporsional. Penelitian menunjukkan bahwa beban tugas yang tidak seimbang dapat berdampak negatif terhadap kualitas pembelajaran dan pengalaman belajar mahasiswa (Baik et al., 2015).

Risiko tersebut tidak bisa dianggap remeh. Ketidaktepatan dalam memperkirakan beban tugas bisa menyebabkan kondisi yang disebut *academic overload*. Misra dan McKean (2000) dalam penelitiannya menemukan bahwa beban akademik yang terlalu tinggi berkorelasi erat dengan meningkatnya tingkat stres mahasiswa dan rendahnya efektivitas mereka dalam mengelola waktu. Temuan ini

menegaskan bahwa persoalan beban tugas tidak semata-mata tergantung pada kemampuan individu mahasiswa, tetapi juga sangat ditentukan oleh kualitas perancangan pembelajaran.

Meskipun isu beban tugas telah lama menjadi perhatian, sistem informasi akademik yang tersedia saat ini umumnya masih berfokus pada fungsi administratif, seperti pencatatan data akademik, pengelolaan nilai, dan informasi perkuliahan. Pemanfaatan teknologi informasi untuk analisis yang lebih mendalam terhadap proses pembelajaran, khususnya dalam memahami isi tugas secara otomatis, masih relatif terbatas. Studi menunjukkan bahwa penerapan kecerdasan buatan dalam pendidikan tinggi sebagian besar masih berorientasi pada analisis data dan belum banyak diarahkan pada pemahaman konteks pembelajaran secara mendalam (Zawacki-Richter et al., 2019). Oleh karena itu, belum banyak sistem yang secara khusus dirancang untuk menganalisis deskripsi tugas secara otomatis guna memperkirakan tingkat kesulitan atau beban kerja secara objektif.

Dari sinilah kemudian muncul gagasan untuk memanfaatkan kecerdasan buatan, terutama pendekatan yang mampu memahami teks secara lebih kontekstual. Salah satu pendekatan yang saat ini dianggap paling relevan adalah *Retrieval-Augmented Generation* (RAG). Secara sederhana, RAG adalah arsitektur kecerdasan buatan yang menggabungkan kemampuan mengambil informasi (*retrieval*) dari sumber pengetahuan eksternal dengan kemampuan menghasilkan teks dari model bahasa berskala besar (*large language models*). RAG memungkinkan pemanfaatan informasi yang relevan sebagai dasar dalam menghasilkan keluaran yang lebih akurat dan kontekstual, dibandingkan dengan pendekatan yang hanya bergantung pada pengetahuan internal model. Dengan demikian, hasil yang dihasilkan menjadi lebih konsisten dan memiliki dasar yang lebih dapat dipertanggungjawabkan (Lewis et al., 2020).

Secara umum, penelitian terkait *Retrieval-Augmented Generation* (RAG) lebih banyak difokuskan untuk menghasilkan keluaran teks berbasis pengetahuan seperti *chatbot*, penilaian dokumen, serta pembuatan konten pembelajaran secara otomatis. Meskipun demikian, perkembangan terbaru menunjukkan bahwa pendekatan RAG juga telah mulai diterapkan dalam tugas estimasi, seperti pada penentuan *story point* dalam pengembangan perangkat lunak, di mana RAG memanfaatkan tugas-tugas historis yang serupa sebagai referensi untuk menghasilkan nilai estimasi secara otomatis (Maha et al., 2026). Selain itu, RAG juga telah digunakan dalam tugas prediksi seperti *time-series forecasting*, yang memanfaatkan data historis untuk meningkatkan akurasi prediksi dan kemampuan generalisasi model (Ning et al., 2025; Tire et al., 2026; Yang et al., 2024). Perkembangan ini mengindikasikan bahwa RAG memiliki potensi yang kuat untuk digunakan dalam tugas estimasi atau prediksi nilai secara lebih kontekstual, khususnya ketika pola dari data sebelumnya dimanfaatkan sebagai dasar pengambilan keputusan. Dalam konteks ini, sistem akan menganalisis deskripsi tugas dengan memanfaatkan data historis tugas yang mirip sebagai referensi untuk menghasilkan estimasi beban tugas.

Agar hasil estimasi beban tugas ini benar-benar berguna, informasi yang dihasilkan tidak cukup hanya disimpan sebagai data mentah. Perlu ada media yang memudahkan pengguna untuk melihat dan memaknai informasi tersebut. *Dashboard* interaktif menjadi pilihan yang tepat karena mampu menyajikan data secara visual, ringkas, dan mudah dipahami sekilas. Melalui *dashboard*, pengguna dapat membandingkan tingkat kompleksitas antar tugas, memantau kemungkinan terjadinya penumpukan beban akademik pada periode tertentu, serta mengevaluasi apakah beban tugas sudah seimbang antar mata kuliah yang diampunya. Dalam penelitian ini, *Streamlit* dipilih sebagai alat untuk membangun *dashboard* karena kemampuannya dalam menyajikan antarmuka interaktif dengan cepat dan dapat diintegrasikan langsung dengan proses analisis berbasis AI.

Berdasarkan uraian di atas, penelitian ini hadir untuk mengisi kesenjangan antara kebutuhan dalam merancang tugas akademik yang proporsional dan terukur, dengan keterbatasan sistem informasi akademik yang ada saat ini. Oleh karena itu, penelitian ini mengambil fokus pada “Pengembangan Dashboard Estimasi Beban Tugas Mahasiswa Berbasis *Retrieval-Augmented Generation* (RAG) (Studi Kasus: Tugas Akademik Sistem Informasi)” sebagai sebuah sistem yang dapat membantu dalam perancangan tugas. Diharapkan, penelitian ini tidak hanya memberikan pendekatan sistematis dalam merancang tugas yang seimbang dan transparan, tetapi juga memperkaya kajian tentang pengembangan sistem informasi akademik berbasis kecerdasan buatan di lingkungan perguruan tinggi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, permasalahan yang diangkat dalam penelitian ini dirumuskan sebagai berikut:

1. Bagaimana implementasi dashboard estimasi beban tugas mahasiswa berbasis *Retrieval-Augmented Generation* (RAG)?
2. Bagaimana kinerja sistem RAG yang dikembangkan dalam menghasilkan estimasi beban tugas akademik?

1.3 Batasan Masalah

Agar penelitian ini lebih terarah dan sesuai dengan tujuan yang diharapkan, maka batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Penelitian ini difokuskan pada tugas akademik mahasiswa Program Studi Sistem Informasi sebagai studi kasus.
2. Pengguna utama sistem adalah dosen, sedangkan sistem digunakan untuk menghasilkan estimasi beban tugas akademik berdasarkan karakteristik tugas yang dianalisis.
3. Input sistem berupa deskripsi teks tugas akademik, tanpa mempertimbangkan faktor non-teks seperti kondisi individu mahasiswa atau metode pengajaran dosen.

4. Estimasi beban tugas yang dihasilkan sistem dibatasi pada rentang tenggat waktu penyelesaian (*deadline*), rentang lama pengerjaan, dan tingkat kesulitan tugas (*difficulty*).
5. Model yang digunakan dalam sistem adalah pendekatan *Retrieval-Augmented Generation* (RAG) dengan memanfaatkan data tugas akademik yang telah dikurasi, yang bersumber dari riwayat tugas akademik serta data sintetis, sebagai basis retrieval.
6. Estimasi beban tugas yang dihasilkan sistem merepresentasikan beban penyelesaian tugas secara keseluruhan sesuai deskripsi tugas yang diberikan, bukan beban kerja individu setiap mahasiswa pada tugas yang dikerjakan secara kelompok.
7. Sistem dikembangkan dalam bentuk dashboard berbasis Streamlit dan dijalankan secara lokal.
8. Evaluasi sistem difokuskan pada kualitas *retrieval*, kualitas hasil estimasi, kualitas *reasoning*, serta konsistensi keluaran sistem, dan tidak mencakup penilaian terhadap efektivitas pedagogis maupun dampak langsung terhadap capaian belajar mahasiswa.
9. Sistem yang dikembangkan dalam penelitian ini masih bersifat *prototipe*. Untuk penggunaan secara luas, diperlukan penyesuaian infrastruktur layanan model bahasa, seperti peningkatan kapasitas API atau mekanisme pengelolaan permintaan, yang tidak dibahas dalam penelitian ini.

1.4 Tujuan Penelitian

Tujuan yang ingin dicapai dalam penelitian ini adalah sebagai berikut:

1. Melakukan implementasi dashboard estimasi beban tugas mahasiswa berbasis *Retrieval-Augmented Generation* (RAG).
2. Melakukan evaluasi terhadap kinerja sistem *Retrieval-Augmented Generation* (RAG) dalam menghasilkan estimasi beban akademik.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Memberikan kontribusi akademis dalam penerapan pendekatan *Retrieval-Augmented Generation* (RAG) pada bidang pendidikan tinggi, khususnya untuk estimasi beban tugas akademik berdasarkan deskripsi tugas dan data historis tugas sebelumnya.
2. Menyediakan sistem yang mampu menghasilkan estimasi beban tugas akademik berupa rentang tenggat waktu penyelesaian, rentang lama pengerjaan, tingkat kesulitan tugas, serta reasoning yang menjelaskan dasar pemberian estimasi.
3. Memberikan informasi estimasi beban tugas yang dapat digunakan sebagai referensi tambahan dalam proses perancangan dan penyusunan tugas akademik pada mata kuliah di Program Studi Sistem Informasi.

4. Menunjukkan potensi pemanfaatan *Retrieval-Augmented Generation* (RAG) tidak hanya untuk menghasilkan keluaran kualitatif seperti jawaban atau penjelasan berbasis dokumen, tetapi juga untuk mendukung proses estimasi yang menghasilkan keluaran kuantitatif berdasarkan data historis yang relevan.
5. Menjadi referensi bagi penelitian selanjutnya dalam pengembangan sistem cerdas di bidang akademik yang memanfaatkan integrasi antara teknik retrieval, model bahasa generatif, dan data historis untuk menghasilkan estimasi yang lebih kontekstual dan dapat dijelaskan.

1.6 Landasan Teori

1.6.1 Sistem Informasi

Sistem informasi merupakan suatu sistem yang terdiri atas sekumpulan komponen yang saling berinteraksi untuk mengumpulkan, mengolah, menyimpan, dan mendistribusikan informasi guna mendukung proses pengambilan keputusan dalam suatu organisasi (Stair & Reynolds, 2010). Dalam operasionalnya, sistem informasi dapat dijelaskan melalui model *input–process–keluaran* (IPO), di mana input berupa data mentah yang dikumpulkan dari berbagai sumber, kemudian diproses menjadi informasi yang memiliki nilai guna, dan menghasilkan keluaran yang digunakan oleh pengguna dalam mendukung aktivitas operasional maupun strategis (Satzinger et al., 2010). Model umum sistem informasi ditunjukkan pada **Gambar 1**.



Gambar 1. Model Umum Sistem Informasi Sumber : Diolah Penulis dari Satzinger et al. (2010)

Secara umum, sistem informasi berfungsi untuk meningkatkan efektivitas dan efisiensi pengelolaan informasi melalui pemanfaatan teknologi komputer, di mana sistem informasi memungkinkan organisasi untuk mengintegrasikan berbagai sumber data, mengolahnya secara terstruktur, serta menyajikan informasi yang relevan dan mudah dipahami oleh pengguna. Dalam prosesnya, sistem informasi tidak hanya berperan sebagai sarana penyimpanan data, tetapi juga sebagai mekanisme yang mengubah data mentah menjadi informasi yang bernilai guna melalui tahapan pengumpulan, pemrosesan, penyimpanan, dan penyajian informasi secara sistematis. Oleh karena itu, sistem informasi telah banyak diterapkan dalam berbagai bidang, termasuk pendidikan, untuk mendukung pengelolaan data dan membantu proses pengambilan keputusan berbasis informasi yang tersedia (Bourgeois et al., 2014).

1.6.2 Estimasi Beban Tugas Akademik (*Academic Workload Estimation*)

Beban tugas akademik (*Academic Workload Estimation*) dapat didefinisikan sebagai jumlah usaha yang harus dilakukan untuk menyelesaikan suatu tugas dalam periode waktu tertentu. Dalam konteks akademik, beban kerja mahasiswa dipengaruhi oleh berbagai faktor seperti kompleksitas tugas, jumlah komponen pekerjaan, serta tingkat kesulitan materi.

Dalam konteks sistem informasi, estimasi merupakan proses memperkirakan nilai atau kondisi tertentu di masa depan berdasarkan data historis, pola yang teridentifikasi, serta model analisis yang digunakan. Estimasi berperan penting dalam berbagai aktivitas pengambilan keputusan, seperti perencanaan sumber daya, penjadwalan, dan evaluasi kinerja sistem. Seiring dengan perkembangan sistem informasi modern, pendekatan estimasi semakin mengarah pada pemanfaatan analisis berbasis data yang didukung oleh teknik statistik dan komputasi. Kualitas estimasi dipengaruhi oleh berbagai faktor, termasuk ketersediaan data yang relevan serta pemilihan metode yang tepat (Jørgensen & Shepperd, 2007).

Estimasi beban tugas bertujuan untuk memprediksi waktu, usaha, dan tingkat kesulitan yang diperlukan dalam menyelesaikan suatu tugas. Estimasi yang akurat sangat penting untuk memastikan bahwa tugas yang diberikan sesuai dengan kemampuan mahasiswa dan tidak menyebabkan beban berlebih. Dalam penelitian ini, estimasi beban kerja mencakup tiga aspek utama, yaitu estimasi tenggat waktu, estimasi lama pengerjaan, dan tingkat kesulitan tugas.

a. Tenggat Waktu (*Deadline*)

Tenggat waktu merupakan batas akhir yang ditetapkan untuk penyelesaian suatu tugas atau penilaian akademik. Misra & Castillo (2004) menemukan bahwa tekanan tenggat waktu yang tinggi berhubungan dengan meningkatnya stres akademik dan kesulitan manajemen waktu mahasiswa. Selanjutnya, Pascoe et al. (2020) menganalisis bahwa tenggat waktu yang ketat dan tidak terkoordinasi antar mata kuliah dapat memperburuk tekanan psikologis mahasiswa, terutama ketika beban tugas menumpuk dalam periode waktu yang berdekatan. Dalam jangka panjang, ketidakseimbangan ini berpotensi menghambat proses pembentukan kompetensi dasar mahasiswa, karena pembelajaran tidak lagi berlangsung secara mendalam, melainkan hanya bersifat reaktif terhadap tuntutan tugas yang menumpuk.

b. Waktu Pengerjaan

Waktu pengerjaan mengacu pada perkiraan durasi yang dibutuhkan mahasiswa untuk menyelesaikan suatu tugas. Kemampuan dalam mengelola dan memperkirakan waktu secara efektif merupakan bagian penting dari strategi belajar mandiri. Penelitian menunjukkan bahwa keterampilan manajemen waktu memiliki hubungan yang signifikan dengan kinerja akademik, di mana pengelolaan waktu yang kurang efektif dapat menyebabkan kesulitan dalam menyelesaikan tugas secara tepat waktu dan menurunkan kualitas hasil belajar (Aeon & Aguinis, 2017). Hal ini menegaskan bahwa pengelolaan waktu

merupakan faktor penting dalam menghadapi beban tugas akademik yang kompleks.

c. **Tingkat Kesulitan Tugas**

Tingkat kesulitan tugas mencerminkan kompleksitas kognitif dan teknis yang terlibat dalam penyelesaian tugas. Dalam konteks pengukuran dan estimasi kesulitan, kompleksitas tugas sering dikaitkan dengan tingkat tuntutan kognitif yang diperlukan, seperti kemampuan memahami, menganalisis, dan mengevaluasi informasi. Clark et al. (2008) menjelaskan bahwa kompleksitas kognitif suatu tugas berkaitan dengan jumlah proses mental yang diperlukan dalam penyelesaiannya, sehingga tugas dengan tuntutan kognitif yang lebih tinggi cenderung memerlukan usaha yang lebih besar. Oleh karena itu, estimasi tingkat kesulitan tugas dapat dilakukan dengan mempertimbangkan kompleksitas instruksi, jumlah langkah penyelesaian, jumlah keluaran, serta tingkat pemrosesan kognitif yang diperlukan.

1.6.3 Kecerdasan Buatan (Artificial Intelligence)

Kecerdasan Buatan (*Artificial Intelligence* atau AI) merupakan cabang ilmu komputer yang berfokus pada pengembangan sistem yang mampu meniru kemampuan kognitif manusia, seperti belajar, memahami, mengambil keputusan, dan menyelesaikan masalah secara otomatis (Russell & Norvig, 2021). AI memungkinkan mesin untuk melakukan analisis terhadap data dan menghasilkan keputusan atau prediksi berbasis pola yang dipelajari dari data tersebut.

Secara konseptual, AI tidak hanya berfungsi untuk otomatisasi, tetapi juga memiliki kemampuan untuk belajar dan beradaptasi berdasarkan data. Rich dan Knight (2009) menjelaskan bahwa AI dirancang untuk mengembangkan sistem yang mampu menyesuaikan diri terhadap kondisi baru. Perkembangan modern AI didukung oleh teknik *machine learning* dan *deep learning* yang memungkinkan sistem mengekstraksi pola kompleks dari data secara lebih akurat dan efisien (Goodfellow et al., 2016).

Dalam bidang pendidikan tinggi, kecerdasan buatan (Artificial Intelligence/AI) dimanfaatkan untuk mendukung analisis data akademik, personalisasi pembelajaran, serta evaluasi otomatis. Penelitian oleh Wayne et al. (2019) menunjukkan bahwa AI memiliki potensi dalam mendukung pengambilan keputusan berbasis data melalui pemanfaatan analitik pembelajaran. Seiring dengan perkembangan tersebut, pemanfaatan AI juga mencakup pendekatan yang berfokus pada pengolahan data tidak terstruktur, seperti teks, melalui *Natural Language Processing* (NLP), serta integrasi dengan model generatif berbasis *Retrieval-Augmented Generation* (RAG) untuk menghasilkan informasi yang lebih kontekstual. Dalam konteks penelitian ini, kedua pendekatan tersebut dimanfaatkan untuk mengolah data teks, mengidentifikasi kompleksitas tugas, serta menghasilkan estimasi yang lebih terukur. Pendekatan ini akan dibahas lebih lanjut pada bagian berikutnya.

1.6.4 Pemrosesan Bahasa Alami (Natural Language Processing)

Pemrosesan Bahasa Alami atau *Natural Language Processing* (NLP) merupakan salah satu cabang kecerdasan buatan yang berfokus pada interaksi antara komputer dan bahasa manusia. Tujuan utama NLP adalah memungkinkan sistem komputasi untuk memahami, memproses, dan menghasilkan bahasa alami secara bermakna (Jurafsky & Martin, 2026).

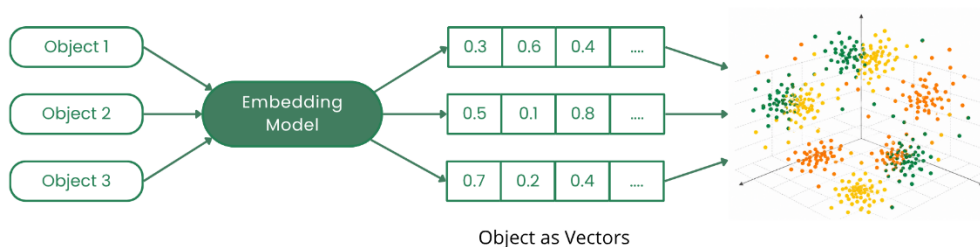
Bird et al. (2009) mendeskripsikan NLP sebagai disiplin yang mengombinasikan linguistik, ilmu komputer, dan statistika untuk memodelkan struktur bahasa alami sehingga dapat dianalisis oleh mesin. Dalam konteks sistem informasi, NLP berperan penting dalam mengolah data teks tidak terstruktur yang jumlahnya semakin meningkat, seperti deskripsi tugas, dokumen akademik, dan laporan naratif.

Teks deskripsi tugas akademik umumnya disusun dalam bentuk naratif bebas yang mengandung variasi kosakata, struktur kalimat, serta tingkat detail yang berbeda-beda. Karakteristik ini menyebabkan informasi penting dalam teks sering kali bersifat implisit dan tidak terstruktur, sehingga sulit diproses menggunakan pendekatan berbasis aturan sederhana. Dalam sistem informasi akademik, *Natural Language Processing* (NLP) digunakan untuk mengekstraksi makna dari deskripsi tugas, seperti tujuan tugas, ruang lingkup pekerjaan, serta aktivitas yang harus dilakukan mahasiswa. Aggarwal (2018) menjelaskan bahwa teknik NLP memungkinkan sistem untuk mengidentifikasi pola semantik dalam teks sehingga informasi penting dapat direpresentasikan dalam bentuk yang lebih terstruktur. Dengan demikian, NLP menjadi komponen kunci dalam menjembatani teks deskripsi tugas yang bersifat kualitatif dengan proses komputasi yang bersifat kuantitatif, seperti estimasi beban tugas akademik.

1.6.5 Representasi Teks dan Embedding

Representasi teks merupakan proses transformasi data teks menjadi bentuk numerik agar dapat diproses oleh komputer. Dalam *Natural Language Processing* (NLP), teks yang bersifat tidak terstruktur perlu dikonversi menjadi representasi vektor sehingga dapat dianalisis secara matematis (Jurafsky & Martin, 2026). Pendekatan ini menjadi dasar dalam berbagai tugas analisis teks, termasuk klasifikasi, pencarian informasi, dan prediksi berbasis teks.

Salah satu metode modern dalam representasi teks adalah *embedding*, yaitu teknik yang mengubah kata, kalimat, atau dokumen menjadi vektor dalam ruang multidimensi. Embedding memungkinkan sistem menangkap hubungan semantik antar teks, sehingga teks dengan makna yang mirip akan memiliki jarak vektor yang berdekatan (Mikolov et al., 2013). Perkembangan embedding berbasis model transformer juga meningkatkan kemampuan representasi konteks secara lebih akurat dalam level kalimat maupun dokumen.



Gambar 2. Ilustrasi proses embedding model

Gambar 2 memperlihatkan bagaimana objek seperti kata, kalimat, atau dokumen direpresentasikan oleh model embedding menjadi vektor numerik dalam ruang berdimensi tinggi. Setiap objek akan dipetakan ke vektor unik yang menangkap karakteristik semantiknya, sehingga objek dengan makna yang mirip akan memiliki posisi yang saling berdekatan dalam ruang vektor. Representasi ini menjadi dasar dalam berbagai tugas pemrosesan bahasa alami, seperti pencarian berbasis kemiripan (*semantic similarity*).

Salah satu model *embedding* berbasis *transformer* yang banyak digunakan dalam tugas *retrieval* dan *semantic similarity* adalah *Multilingual-E5-Base*. *Multilingual-E5-Base* merupakan model *teks embedding* berbasis *transformer* yang dirancang untuk menghasilkan representasi vektor semantik dari teks dalam berbagai bahasa. Model ini merupakan bagian dari keluarga E5 (*Embedding from Bidirectional Encoder Representations*) yang dikembangkan untuk mendukung berbagai tugas *retrieval*, *semantic similarity*, *clustering*, dan klasifikasi teks. Model ini dibangun di atas *arsitektur transformer* dan dilatih menggunakan pendekatan *contrastive learning* sehingga teks yang memiliki makna serupa akan direpresentasikan dalam vektor yang berdekatan pada ruang embedding (Wang et al., 2024).

Berbeda dengan metode representasi teks tradisional seperti *Bag of Words* atau TF-IDF yang berfokus pada kemunculan kata, *multilingual-E5-Base* mampu menangkap hubungan semantik dan konteks antar kalimat. Model ini mendukung puluhan bahasa, termasuk bahasa Indonesia, sehingga dapat digunakan untuk melakukan pencarian dokumen berdasarkan kesamaan makna meskipun terdapat perbedaan struktur kalimat maupun pemilihan kata (Wang et al., 2024).

Proses pembelajaran *multilingual-E5-Base* dilakukan melalui dua tahap utama, yaitu *contrastive pre-training* pada sejumlah besar pasangan teks multibahasa dan *supervised fine-tuning* menggunakan *dataset retrieval* berlabel. Pendekatan tersebut memungkinkan model menghasilkan *embedding* yang efektif untuk tugas *semantic retrieval* dan *retrieval-augmented generation* (RAG). Evaluasi yang dilakukan oleh Wang dkk. menunjukkan bahwa keluarga *multilingual-E5* memiliki performa yang kompetitif pada berbagai benchmark *retrieval* multibahasa dan *semantic similarity* (Wang et al., 2024).

1.6.6 Semantic Similarity dan Cosine Similarity

Semantic Similarity (kesamaan semantik) merupakan konsep yang digunakan untuk mengukur tingkat kesamaan makna antara dua kata, frasa, teks, kalimat, atau dokumen. Berbeda dengan pendekatan berbasis kesamaan kata (*lexical similarity*), *semantic similarity* mempertimbangkan konteks serta makna keseluruhan kalimat, sehingga mampu mengidentifikasi hubungan semantik meskipun teks menggunakan kata-kata yang berbeda (Jurafsky & Martin, 2026). Pendekatan ini banyak digunakan pada berbagai tugas *Natural Language Processing* (NLP), seperti *information retrieval*, *question answering*, dan *document matching*.

Pada sistem berbasis *embedding*, *semantic similarity* dihitung dengan merepresentasikan teks ke dalam bentuk vektor numerik. Setelah setiap teks direpresentasikan sebagai vektor, tingkat kemiripan antar teks dapat diukur menggunakan berbagai metode, salah satunya adalah *cosine similarity*. *Cosine similarity* merupakan metode pengukuran kemiripan yang digunakan untuk menghitung kesamaan antara dua vektor berdasarkan sudut di antara keduanya dalam ruang vektor. Konsep ini banyak digunakan dalam *information retrieval* dan *machine learning* (Manning et al., 2009). Secara matematis, cosine similarity antara dua vektor x dan y dengan dimensi n dapat dirumuskan pada persamaan (1) berikut:

$$\text{cosine similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}} \quad (1)$$

Di mana:

n = Dimensi dari vektor (jumlah fitur/atribut).

A_i = Nilai dari vektor A pada dimensi ke- i

B_i = Nilai dari vektor B pada dimensi ke- i

Nilai *cosine similarity* berada pada rentang -1 hingga 1. Semakin mendekati nilai 1, semakin tinggi tingkat kemiripan semantik antara kedua teks. Dalam praktik representasi teks berbasis *embedding*, nilai *cosine similarity* umumnya berada pada rentang 0 hingga 1 karena seluruh vektor merepresentasikan informasi semantik yang searah.

Dalam konteks sistem berbasis *embedding*, *cosine similarity* digunakan untuk mengukur kemiripan antara deskripsi tugas baru dengan tugas-tugas historis yang telah tersimpan dalam basis data. Nilai kemiripan tersebut digunakan pada proses *retrieval* untuk menemukan tugas yang paling relevan sebagai konteks bagi pendekatan *Retrieval-Augmented Generation* (RAG). Dengan demikian, sistem dapat memanfaatkan referensi historis yang memiliki makna paling dekat dengan tugas yang sedang dianalisis sehingga estimasi beban tugas yang dihasilkan menjadi lebih kontekstual.

1.6.7 Vector Database dan FAISS

Vector database merupakan sistem penyimpanan data yang dirancang untuk menyimpan dan mengelola data dalam bentuk representasi vektor numerik. Berbeda dengan basis data relasional yang menggunakan pencarian berbasis nilai atau kata kunci, *vector database* memungkinkan pencarian berbasis kemiripan (*similarity search*) dalam ruang berdimensi tinggi melalui pendekatan *nearest neighbor search*. Pendekatan ini banyak digunakan untuk menangani data tidak terstruktur seperti teks yang telah direpresentasikan dalam bentuk *embedding* (Johnson et al., 2021). Konsep pencarian berbasis ruang vektor ini juga sejalan dengan pendekatan *information retrieval* yang dijelaskan oleh Manning et al. (2009), di mana kemiripan antar dokumen dihitung berdasarkan kedekatan vektor dalam ruang semantik.

Dalam *vector database*, setiap data direpresentasikan sebagai vektor dalam ruang multidimensi, sehingga hubungan antar data dapat diukur berdasarkan jarak atau kedekatan antar vektor. Pendekatan ini memungkinkan sistem untuk menemukan data yang memiliki makna serupa meskipun tidak memiliki kesamaan kata secara langsung. Oleh karena itu, *vector database* banyak digunakan dalam sistem berbasis *semantic retrieval*, yang membutuhkan pencarian informasi berdasarkan makna, bukan sekadar kecocokan kata.

Salah satu implementasi yang banyak digunakan adalah FAISS (*Facebook AI Similarity Search*), yang dikembangkan oleh Meta AI untuk melakukan pencarian *nearest neighbor* secara efisien pada data berdimensi tinggi (Johnson et al., 2021). FAISS dirancang untuk mengoptimalkan proses pencarian vektor dengan memanfaatkan struktur indeks yang mampu mempercepat perhitungan jarak antar data. Kemampuan ini menjadikan FAISS sangat sesuai untuk mendukung sistem yang memerlukan pencarian konteks relevan secara cepat, seperti pada pendekatan *Retrieval-Augmented Generation* (RAG), di mana data historis yang paling mirip digunakan sebagai referensi dalam proses analisis dan estimasi.

1.6.8 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) merupakan pendekatan dalam kecerdasan buatan yang menggabungkan mekanisme *information retrieval* dengan kemampuan generasi teks menggunakan *Large Language Model* (LLM). Pendekatan ini memungkinkan sistem menghasilkan informasi yang tidak hanya koheren secara linguistik, tetapi juga didukung oleh data eksternal yang relevan sehingga meningkatkan akurasi dan keandalan keluaran (Lewis et al., 2020).

RAG dirancang untuk mengatasi keterbatasan model generatif murni yang hanya bergantung pada pengetahuan yang tersimpan dalam parameter model. Model generatif seperti LLM memiliki keterbatasan dalam mengakses informasi terbaru (*knowledge cutoff*) serta berpotensi menghasilkan informasi yang tidak akurat (*hallucination*). Dengan memanfaatkan proses pencarian informasi dari basis data eksternal, RAG mampu menghasilkan jawaban yang lebih kontekstual, faktual, dan dapat ditelusuri sumbernya (Lewis et al., 2020).

Selain itu, pendekatan RAG juga memberikan fleksibilitas dalam memperbarui pengetahuan sistem tanpa perlu melakukan pelatihan ulang model secara keseluruhan. Hal ini menjadikan RAG sebagai solusi yang efisien dalam pengembangan sistem berbasis kecerdasan buatan yang membutuhkan informasi dinamis dan spesifik domain.

1.6.8.1 Komponen Retrieval-Augmented Generation

RAG terdiri dari tiga komponen utama, yaitu *retrieval*, *augmentation*, dan *generation*, yang bekerja secara terintegrasi dalam menghasilkan keluaran sistem.

a. *Retrieval* (Pengambilan Informasi)

Komponen *retrieval* berfungsi untuk mengambil dokumen atau informasi yang paling relevan dari basis pengetahuan berdasarkan masukan pengguna. Proses ini umumnya dilakukan melalui representasi vektor (*embedding*) yang memungkinkan pencarian semantik, bukan sekadar pencocokan kata kunci. Lewis et al. (2020) menjelaskan bahwa tahap *retrieval* memiliki peran krusial dalam menentukan kualitas keluaran sistem, karena informasi yang diambil menjadi konteks utama yang digunakan dalam proses generasi jawaban selanjutnya.

b. *Augmentation* (Pengayaan Informasi)

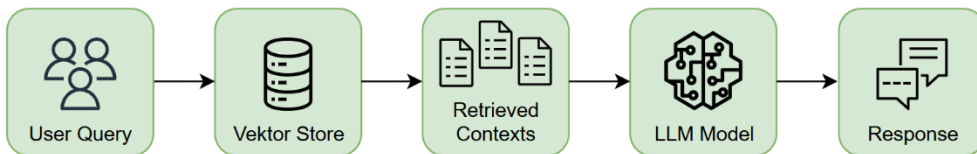
Augmentation adalah proses menggabungkan hasil *retrieval* sebagai konteks tambahan yang akan digunakan oleh model generatif. Informasi yang diperoleh dari basis data disusun sedemikian rupa menjadi sebuah *prompt* agar dapat memperkaya pemahaman model terhadap query yang diberikan

c. *Generation* (Generasi Jawaban)

Generation merupakan tahap di mana model bahasa menghasilkan keluaran berupa teks berdasarkan query dan konteks tambahan yang telah diperoleh. Dengan adanya konteks dari hasil *retrieval*, jawaban yang dihasilkan menjadi lebih akurat, relevan, dan kontekstual (Izacard & Grave, 2021).

Selain ketiga komponen utama tersebut, dalam implementasi *Retrieval-Augmented Generation* (RAG) juga terdapat tahap *indexing*, yaitu proses pengubahan dan penyimpanan data ke dalam bentuk representasi vektor pada basis data vektor agar dapat diakses secara efisien pada tahap *retrieval*. Tahap ini umumnya dilakukan pada fase pra-pemrosesan (*offline indexing phase*) sebelum sistem digunakan. Proses ini mendukung efisiensi pencarian kemiripan pada ruang vektor melalui teknik *approximate nearest neighbor search* yang digunakan dalam sistem seperti FAISS (Johnson et al., 2021).

1.6.8.2 Alur Proses Retrieval-Augmented Generation



Gambar 3. Alur Proses RAG Sumber : Diolah penulis dari Lewis et al. (2020)

Secara konseptual, alur proses RAG sederhana ditunjukkan pada **Gambar 3**. Proses dimulai dari masukan berupa teks deskriptif yang diberikan oleh pengguna (*Pengguna Query*). Masukan ini kemudian diproses oleh komponen *retriever mechanism*, yang bertugas untuk mencari informasi yang relevan dari basis pengetahuan. Untuk memungkinkan pencarian semantik, data dalam basis pengetahuan sebelumnya telah dikonversi menjadi representasi vektor menggunakan *embedding model* dan disimpan dalam *vector store*. Selain itu, basis pengetahuan yang digunakan dapat berasal dari sumber lokal atau data spesifik domain (*proprietary local knowledge base*), sehingga sistem dapat menghasilkan jawaban yang lebih kontekstual.

Setelah proses *retrieval* dilakukan, sistem akan menghasilkan sejumlah data dengan tingkat relevansi tinggi (*retrieved contexts*). Data ini kemudian digunakan sebagai konteks tambahan yang diberikan kepada LLM. Pada tahap ini, LLM yang telah dilatih menggunakan dataset besar (*pretrained on large general dataset*) akan memproses masukan pengguna bersama dengan konteks hasil *retrieval* untuk menghasilkan jawaban.

Hasil akhir dari proses ini adalah *response* yang dikirimkan kembali kepada pengguna. Dengan adanya integrasi antara data eksternal dan kemampuan generatif LLM, sistem RAG mampu menghasilkan jawaban yang tidak hanya koheren secara bahasa, tetapi juga lebih akurat dan berbasis informasi yang relevan.

1.6.9 Large Language Model (LLM)

Large Language Model (LLM) merupakan model kecerdasan buatan berbasis pembelajaran mendalam yang dilatih menggunakan data teks berskala besar untuk memahami, memodelkan, dan menghasilkan bahasa alami. LLM dirancang untuk menangkap pola sintaksis, semantik, serta hubungan kontekstual antarkata dalam teks melalui mekanisme representasi vektor berdimensi tinggi. Brown et al. (2020) mendeskripsikan LLM sebagai model bahasa dengan jumlah parameter sangat besar yang mampu melakukan berbagai tugas linguistik tanpa pelatihan khusus pada setiap tugas tersebut.

Dalam perkembangannya, *Large Language Model* (LLM) banyak digunakan dalam sistem tanya jawab, ringkasan dokumen, analisis teks, serta asisten cerdas

di berbagai domain, termasuk pendidikan. Brown et al. (2020) menjelaskan bahwa kemampuan utama LLM terletak pada pemodelan konteks bahasa dalam skala besar, sehingga model mampu memahami hubungan antar kata dalam suatu teks secara menyeluruh, bukan hanya secara terpisah. Kemampuan ini memungkinkan LLM untuk digunakan dalam berbagai tugas bahasa alami tanpa pelatihan khusus untuk setiap tugas.

Meskipun memiliki kemampuan yang tinggi, LLM memiliki keterbatasan seperti ketergantungan pada data pelatihan dan potensi menghasilkan informasi yang tidak akurat. Oleh karena itu, dalam penelitian ini LLM dikombinasikan dengan pendekatan RAG untuk meningkatkan kualitas hasil estimasi.

1.6.10 Prompt Engineering

Prompt engineering merupakan teknik dalam mengarahkan perilaku *Large Language Model* (LLM) melalui perancangan instruksi atau *prompt* yang terstruktur dan jelas. Prompt berfungsi sebagai antarmuka utama antara pengguna dan model, sehingga kualitas prompt sangat menentukan kualitas keluaran yang dihasilkan. Prompt yang dirancang dengan baik mampu meningkatkan akurasi, konsistensi, serta relevansi respons model terhadap konteks permasalahan (Liu et al., 2023).

Secara konseptual, *prompt engineering* tidak hanya berfokus pada pemberian instruksi sederhana, tetapi juga mencakup strategi penyusunan konteks, format pertanyaan, serta penyajian informasi tambahan agar model dapat memahami tugas secara lebih tepat. Brown et al. (2020) menunjukkan bahwa model bahasa besar memiliki kemampuan *few-shot* dan *zero-shot learning*, di mana performa model dapat ditingkatkan secara signifikan melalui perancangan prompt tanpa perlu pelatihan ulang model.

Dalam konteks *Retrieval-Augmented Generation* (RAG), *prompt engineering* berperan penting dalam mengintegrasikan hasil *retrieval* ke dalam proses generasi. Informasi yang diperoleh dari basis pengetahuan disusun sebagai konteks tambahan (*konteks injection*) dan digabungkan dengan instruksi analisis dalam prompt. Dengan pendekatan ini, model tidak hanya menghasilkan teks berdasarkan pengetahuan umum, tetapi juga mempertimbangkan data historis yang relevan, sehingga estimasi yang dihasilkan menjadi lebih terstruktur, kontekstual, dan sesuai dengan kebutuhan analisis beban tugas mahasiswa.

1.6.11 Data Sintetis dalam Pengembangan Sistem AI

Data sintetis merupakan data yang dihasilkan secara artifisial menggunakan teknik komputasi untuk mensimulasikan karakteristik data nyata. Dalam konteks pembelajaran mesin dan pengembangan sistem kecerdasan buatan, data sintetis digunakan untuk mengatasi keterbatasan ketersediaan data, memperkaya variasi dataset, serta mendukung perlindungan privasi data asli (Jordon et al., 2022). Data ini menjadi pendekatan penting dalam pengembangan model kecerdasan buatan

modern, terutama pada kondisi ketika data asli sulit diperoleh atau memiliki keterbatasan akses.

Nikolenko (2019) menyimpulkan bahwa data sintetis yang dihasilkan melalui pendekatan generatif dapat memperluas cakupan pola data dan berpotensi meningkatkan kemampuan generalisasi model, selama distribusi data sintetis mampu merepresentasikan karakteristik data nyata yang ingin dimodelkan secara akurat.

1.6.12 Evaluasi Sistem RAG

Evaluasi merupakan tahapan penting dalam pengembangan sistem *Retrieval-Augmented Generation* (RAG) untuk memastikan bahwa setiap komponen dalam arsitektur bekerja secara efektif dan menghasilkan keluaran yang berkualitas. Berbeda dengan sistem *machine learning* konvensional yang umumnya hanya mengevaluasi hasil prediksi akhir, evaluasi pada sistem RAG perlu dilakukan secara multidimensi karena melibatkan dua komponen utama, yaitu *retrieval* dan *generation*. Kinerja *retrieval* menentukan kualitas konteks yang diberikan kepada model bahasa, sedangkan kinerja *generation* menentukan kualitas keluaran yang dihasilkan berdasarkan konteks tersebut. Oleh karena itu, evaluasi sistem RAG umumnya mencakup aspek relevansi hasil *retrieval*, akurasi keluaran, kualitas *reasoning*, *faithfulness* terhadap konteks, serta konsistensi format keluaran yang dihasilkan (Yu et al., 2025).

Menurut Es et al. (2024), evaluasi sistem RAG tidak dapat dilakukan hanya dengan satu metrik karena kualitas sistem dipengaruhi oleh kemampuan menemukan informasi yang relevan, kemampuan model memanfaatkan informasi tersebut secara tepat, serta kualitas keluaran yang dihasilkan. Oleh sebab itu, berbagai metrik evaluasi digunakan untuk mengukur setiap komponen secara terpisah sehingga kelemahan sistem dapat diidentifikasi secara lebih spesifik.

1.6.12.1 Evaluasi Retrieval dengan Hit Rate dan Precision Rate

Evaluasi *retrieval* merupakan tahapan penting dalam sistem *Retrieval-Augmented Generation* (RAG), karena kualitas informasi yang diambil dari basis pengetahuan akan sangat memengaruhi akurasi keluaran yang dihasilkan oleh *Large Language Model* (LLM). Dalam arsitektur RAG, tahap *retrieval* berfungsi sebagai penyedia konteks utama, sehingga keberhasilan sistem dalam menemukan dan memilih informasi yang relevan menjadi faktor kunci dalam menentukan performa keseluruhan sistem (Lewis et al., 2020).

Untuk mengukur performa *retrieval*, digunakan metrik evaluasi yang mampu menilai relevansi hasil pencarian terhadap *query* pengguna. Dua metrik yang umum digunakan adalah *Hit Rate* dan *Precision*, yang masing-masing memiliki peran dalam mengukur keberhasilan dan kualitas hasil *retrieval*.

a. Hit Rate

Hit Rate (atau *Success@K*) merupakan metrik evaluasi yang umum digunakan dalam *information retrieval* dan *recommender systems* untuk mengukur kemampuan sistem dalam menemukan setidaknya satu data yang relevan dalam hasil retrieval. Metrik ini bersifat biner, yaitu bernilai 1 apabila terdapat minimal satu hasil yang relevan dalam sejumlah hasil teratas (*jumlah hasil retrieval*), dan bernilai 0 apabila tidak ditemukan hasil yang relevan. Konsep evaluasi berbasis peringkat ini sejalan dengan pendekatan dalam *information retrieval* yang membahas pengukuran kualitas hasil pencarian berdasarkan relevansi dokumen (Manning et al., 2009).

Hit Rate dapat didefinisikan sebagai metrik evaluasi yang mengukur *proporsi query* yang memiliki setidaknya satu hasil relevan dalam daftar hasil *tugas hasil retrieval*. Secara matematis, *Hit Rate* dapat dirumuskan sebagai berikut:

$$\text{Hit rate} = \frac{\text{Jumlah query dengan minimal satu hasil relevan}}{\text{Total jumlah query}} \quad (2)$$

Nilai *Hit Rate* berada pada rentang 0 hingga 1. Semakin tinggi nilai *Hit Rate*, semakin baik kemampuan sistem dalam menemukan informasi yang relevan.

Dalam konteks RAG, *Hit Rate* memiliki peran penting karena memastikan bahwa konteks yang diberikan kepada LLM mengandung informasi yang relevan, meningkatkan peluang model menghasilkan jawaban yang akurat, dan memberikan gambaran awal keberhasilan sistem *retrieval*.

b. Precision Rate

Precision merupakan metrik yang digunakan untuk mengukur proporsi data yang relevan dibandingkan dengan seluruh data yang diambil oleh sistem. Metrik ini memberikan gambaran mengenai kualitas hasil *retrieval*, yaitu seberapa banyak hasil yang benar-benar relevan terhadap query pengguna (Manning et al., 2009).

Secara matematis, *Precision* dapat dirumuskan sebagai berikut:

$$\text{Precision} = \frac{\text{Jumlah data relevan yang diambil}}{\text{Total data yang diambil}} \quad (3)$$

Rumus tersebut menunjukkan bahwa nilai *Precision* dihitung berdasarkan perbandingan antara jumlah dokumen relevan yang berhasil diambil dengan total dokumen yang dihasilkan oleh sistem. Nilai *Precision* berada pada rentang 0 hingga 1, di mana nilai yang lebih tinggi menunjukkan kualitas hasil *retrieval* yang lebih baik.

1.6.12.2 Evaluasi Manual Judgement dengan Skala Likert

Manual judgement merupakan metode evaluasi yang dilakukan dengan melibatkan penilai manusia untuk memberikan penilaian terhadap kualitas suatu sistem berdasarkan persepsi, pengalaman, dan pertimbangan subjektif. Dalam penelitian sistem berbasis kecerdasan buatan, *manual judgement* sering digunakan untuk mengevaluasi aspek yang sulit diukur secara kuantitatif, seperti kewajaran hasil, kualitas rekomendasi, keterbacaan, dan kesesuaian keluaran sistem dengan kebutuhan pengguna. Pendekatan ini memungkinkan peneliti memperoleh gambaran mengenai tingkat penerimaan dan kelayakan hasil sistem dari perspektif pengguna secara langsung.

Salah satu instrumen yang umum digunakan dalam manual judgement adalah Skala Likert. Skala Likert merupakan skala psikometrik yang digunakan untuk mengukur sikap, persepsi, atau tingkat persetujuan responden terhadap suatu pernyataan. Skala ini pertama kali diperkenalkan oleh Rensis Likert dan hingga saat ini menjadi salah satu metode pengukuran yang paling banyak digunakan dalam penelitian sosial, pendidikan, maupun teknologi. Skala Likert memungkinkan peneliti mengubah persepsi subjektif responden menjadi data numerik yang dapat dianalisis secara sistematis.

Menurut Joshi et al. (2015), Skala Likert umumnya menggunakan rentang penilaian 1–5 atau 1–7, di mana setiap nilai merepresentasikan tingkat persetujuan atau kualitas tertentu. Pada skala 1–5, interpretasi nilai dapat disajikan seperti pada **Tabel 1**.

Tabel 1. Interpretasi Skala Likert 1–5

Skor	Interpretasi
1	Sangat Tidak Setuju / Sangat Buruk
2	Tidak Setuju / Buruk
3	Netral / Cukup
4	Setuju / Baik
5	Sangat Setuju / Sangat Baik

1.6.12.3 Evaluasi LLM-as-a-Judge

LLM-as-a-Judge merupakan pendekatan evaluasi yang memanfaatkan *Large Language Model* (LLM) sebagai evaluator untuk menilai kualitas keluaran yang dihasilkan oleh model lain atau sistem berbasis kecerdasan buatan. Pendekatan ini berkembang karena evaluasi berbasis manusia membutuhkan waktu, biaya, dan sumber daya yang relatif besar, terutama ketika jumlah data yang dievaluasi cukup banyak. Dengan kemampuan memahami konteks bahasa alami, LLM dapat digunakan untuk memberikan penilaian terhadap kualitas jawaban, konsistensi, relevansi, logika penalaran, maupun kesesuaian keluaran terhadap kriteria tertentu (Zheng et al., 2023).

Meskipun menunjukkan potensi yang besar, beberapa penelitian mengungkapkan bahwa *LLM-as-a-Judge* masih memiliki sejumlah keterbatasan. Gu et al. (2026) menjelaskan bahwa hasil evaluasi dapat dipengaruhi oleh berbagai jenis bias, seperti *position bias*, *verbosity bias*, dan *preference bias* terhadap model tertentu. Selain itu, kualitas penilaian juga sangat bergantung pada desain prompt evaluasi yang digunakan. Oleh karena itu, hasil evaluasi *LLM-as-a-Judge* sebaiknya digunakan sebagai pelengkap evaluasi kuantitatif dan evaluasi pengguna, sehingga diperoleh gambaran yang lebih komprehensif mengenai kualitas sistem yang dikembangkan.

1.6.12.4 Evaluasi Validasi JSON dengan Parsing Success Rate (PSR)

Kemampuan menghasilkan keluaran terstruktur (*structured keluaran*) merupakan salah satu aspek penting dalam implementasi *Large Language Model* (LLM) pada sistem yang membutuhkan integrasi dengan aplikasi lain. Berbeda dengan keluaran berbentuk teks bebas, keluaran terstruktur mengharuskan model mengikuti format tertentu, seperti JSON, XML, atau skema data lainnya sehingga dapat diproses secara otomatis oleh sistem. Keluaran yang tidak sesuai dengan format yang ditentukan dapat menyebabkan kegagalan pemrosesan, meskipun informasi yang terkandung di dalamnya benar secara semantik (Tang et al., 2024).

Penelitian mengenai kemampuan LLM dalam menghasilkan keluaran terstruktur menunjukkan bahwa kepatuhan terhadap format (*schema compliance*) merupakan tantangan tersendiri dalam pengembangan sistem berbasis LLM. Shorten et al. (2024) melalui *benchmark Structured*, RAG menemukan bahwa keberhasilan model dalam menghasilkan keluaran JSON yang valid sangat dipengaruhi oleh kompleksitas tugas dan strategi prompting yang digunakan. Pada beberapa kasus, model dapat menghasilkan informasi yang relevan namun gagal memenuhi format keluaran yang dipersyaratkan sehingga tidak dapat diproses secara otomatis oleh aplikasi.

Selain validitas struktur, reliabilitas keluaran juga menjadi faktor penting dalam implementasi sistem berbasis LLM. Tenckhoff et al. (2026) menjelaskan bahwa evaluasi keluaran terstruktur tidak hanya berfokus pada akurasi isi, tetapi juga pada validitas dokumen dan kemampuan model menghasilkan struktur JSON yang dapat diparsing secara konsisten. Keluaran yang konsisten memungkinkan integrasi yang lebih andal.

1.6.13 Dashboard sebagai Media Visualisasi Data

Dashboard merupakan antarmuka visual yang digunakan untuk menyajikan informasi secara ringkas, terstruktur, dan mudah dipahami sehingga mendukung proses pemantauan dan pengambilan keputusan. Dashboard umumnya menampilkan data dalam bentuk visual seperti grafik, tabel, dan indikator kinerja yang memungkinkan pengguna memahami informasi secara cepat, efisien, dan intuitif tanpa harus menganalisis data mentah secara langsung (Few, 2006). Selain

itu, penggunaan visualisasi dalam dashboard membantu dalam menyederhanakan data yang kompleks menjadi bentuk yang lebih mudah dipahami. Lebih lanjut, dashboard juga berfungsi sebagai alat untuk memonitor, menganalisis, dan mengevaluasi data, baik secara real-time maupun periodik, sehingga dapat mendukung peningkatan kualitas analisis dan pemahaman informasi dalam suatu organisasi (Velcu-laitinen & Yigitbasioglu, 2012).

Secara umum, dashboard dapat diklasifikasikan menjadi beberapa jenis, yaitu dashboard operasional, dashboard analitik, dan dashboard strategis. Dashboard operasional digunakan untuk pemantauan aktivitas harian secara langsung dan biasanya berfokus pada kondisi sistem yang berjalan saat itu, dashboard analitik berfokus pada eksplorasi serta analisis data untuk memahami pola, hubungan, atau informasi tertentu dari data yang tersedia, sedangkan dashboard strategis digunakan untuk mendukung pengambilan keputusan pada tingkat manajemen dengan cakupan informasi yang lebih luas dan bersifat ringkasan (Few, 2006). Masing-masing jenis dashboard memiliki tujuan dan karakteristik yang berbeda dalam penyajian informasi. Dalam konteks penelitian ini, dashboard yang dikembangkan termasuk dalam kategori dashboard analitik karena digunakan untuk menganalisis teks tugas akademik serta menyajikan hasil estimasi beban kerja mahasiswa dalam bentuk informasi yang terstruktur, sehingga dapat membantu pengguna dalam memahami kompleksitas tugas secara lebih cepat dan sistematis.

1.6.14 Streamlit sebagai Framework Dashboard

Streamlit adalah framework *open-source* berbasis Python yang dirancang untuk mempermudah pembuatan aplikasi web interaktif, terutama dalam konteks visualisasi data dan prototyping aplikasi berbasis Python tanpa perlu menguasai HTML, CSS, atau JavaScript. Framework ini memungkinkan pengguna membangun antarmuka yang responsif dan *real-time* langsung dari skrip Python dengan sedikit baris kode, sehingga mempercepat proses pengembangan aplikasi serta mengurangi kompleksitas integrasi *frontend* dan *backend* (Syafarina & Zaenuddin, 2023).

Dalam penelitian ini, Streamlit digunakan untuk membangun dashboard yang mengintegrasikan seluruh proses mulai dari input deskripsi tugas hingga menampilkan hasil estimasi beban tugas mahasiswa. Penggunaan Streamlit mempermudah pengembangan sistem serta meningkatkan interaktivitas pengguna melalui tampilan antarmuka yang intuitif dan dinamis.

1.6.15 LLaMa 3.1 8B

LLaMa 3.1 8B merupakan salah satu model *Large Language Model* (LLM) dari keluarga LLaMa (*Large Language Model Meta AI*) yang dikembangkan oleh Meta. Model ini dirancang untuk menghasilkan teks kontekstual menggunakan arsitektur Transformer, yang pertama kali diperkenalkan oleh Vaswani et al. (2017) dan menjadi fondasi utama model bahasa modern seperti GPT dan BERT. LLaMa 3.1 8B

memiliki sekitar 8 miliar parameter dan dilatih menggunakan lebih dari 15 triliun token data yang mencakup berbagai bahasa dan domain pengetahuan. Dibandingkan generasi sebelumnya, model ini memiliki kemampuan pemahaman konteks yang lebih baik serta mendukung pemrosesan konteks hingga 128.000 token, sehingga mampu menangani input yang lebih panjang dan kompleks.

LLaMa 3.1 8B dirancang untuk memberikan keseimbangan antara performa dan efisiensi komputasi. Meskipun memiliki ukuran yang lebih kecil dibandingkan varian LLaMa 3.1 lainnya, model ini tetap mampu menghasilkan respons yang relevan dan koheren pada berbagai tugas pemrosesan bahasa alami, seperti ringkasan teks, tanya jawab, klasifikasi, dan penalaran berbasis konteks. Karakteristik tersebut menjadikan LLaMa 3.1 8B sesuai untuk digunakan pada sistem *Retrieval-Augmented Generation* (RAG), di mana model harus mampu memanfaatkan informasi hasil retrieval untuk menghasilkan keluaran yang relevan dan kontekstual.

BAB II

METODE PENELITIAN

2.3 Instrumen Penelitian

Instrumen penelitian digunakan sebagai pendukung dalam proses perancangan, pengembangan, dan evaluasi sistem. Instrumen penelitian terdiri atas perangkat keras (*hardware*) dan perangkat lunak (*software*) yang digunakan selama tahapan pengembangan dan evaluasi sistem.

2.3.1 Perangkat Keras (Hardware)

Perangkat keras berupa laptop digunakan dalam proses perancangan dan evaluasi sistem, dengan spesifikasi sebagaimana ditampilkan pada **Tabel 3**.

Tabel 3. Spesifikasi Perangkat Keras

Komponen	Spesifikasi
Sistem Operasi	Windows 11 Home
Processor	AMD Ryzen 7 5700U
RAM	16.0 GB
Penyimpanan	954 GB/SSD 1 TB
GPU	AMD Radeon(TM) Graphics

2.3.2 Perangkat Lunak (Software)

Perangkat lunak yang digunakan dalam penelitian ini meliputi berbagai aplikasi dan *library* yang mendukung proses pengembangan, pengujian, serta implementasi sistem estimasi beban tugas akademik berbasis *Retrieval-Augmented Generation* (RAG). Perangkat lunak tersebut digunakan pada tahapan pengolahan data, pembangunan model retrieval, integrasi model bahasa generatif, pengembangan antarmuka pengguna, hingga proses evaluasi sistem. Penggunaan perangkat lunak yang beragam bertujuan untuk memastikan seluruh komponen sistem dapat berjalan secara terintegrasi, mulai dari proses embedding, pencarian referensi historis, generasi estimasi, hingga penyajian hasil melalui dashboard. Daftar perangkat lunak yang digunakan dalam penelitian ini ditunjukkan pada **Tabel 4**.

Tabel 4. Daftar perangkat lunak dan pustaka yang digunakan dalam pengembangan sistem

No	Nama Perangkat Lunak	Fungsi dalam Penelitian
1	Sistem Operasi Windows	Digunakan sebagai sistem operasi utama dalam pengembangan dan pengujian sistem
2	Python	Bahasa pemrograman utama untuk membangun sistem prediksi beban tugas berbasis RAG

No	Nama Perangkat Lunak	Fungsi dalam Penelitian
3	Visual Studio Code	Lingkungan pengembangan (IDE) untuk penulisan dan pengelolaan kode program
4	Google Forms	Digunakan untuk mengumpulkan dan menyusun data contoh tugas akademik sebagai bagian dari dataset penelitian serta evaluasi kelayakan pengguna
5	Microsoft Excel / CSV Tools	Digunakan untuk penyimpanan, pembersihan, dan pengelolaan dataset sebelum diproses lebih lanjut
6	ChatGPT	Digunakan sebagai alat bantu pembuatan data sintetis berdasarkan contoh teks tugas dari dosen dan model pada evaluasi <i>LLM-as-a-Judge</i>
7	Claude	Digunakan sebagai alternatif alat bantu pembuatan data sintetis untuk memperkaya variasi dataset
8	Google Colaboratory	Digunakan sebagai lingkungan tambahan untuk melakukan Preprocessing dataset.
9	Pandas	Digunakan untuk menyajikan data dalam bentuk tabel pada tahap preprocessing, implementasi (Tugas hasil retrieval), dan evaluasi
10	Sentence Transformers	Digunakan untuk menghasilkan embedding teks yang akan digunakan dalam proses pencarian semantik
11	HuggingFace Transformers	Digunakan untuk mengakses dan memanfaatkan model embedding serta model NLP pendukung
12	NumPy	Digunakan untuk operasi numerik dan perhitungan matematis dalam proses embedding dan similarity
13	FAISS	Library vector database untuk menyimpan dan melakukan pencarian semantik dokumen tugas akademik
14	Pickle	Digunakan untuk menyimpan dan memuat model atau data embedding yang telah diproses
15	Streamlit	Framework untuk membangun antarmuka aplikasi berbasis web sebagai media interaksi pengguna dengan sistem
16	PDFPlumber	Digunakan untuk membaca dan mengekstrak teks dari file PDF yang diunggah oleh pengguna sebagai input sistem
17	Python-docx	Digunakan untuk membaca dan mengekstrak teks dari file DOCX sebagai input sistem
18	Regex(re)	Digunakan untuk membersihkan dan memperbaiki format teks hasil ekstraksi (misalnya dari PDF) dan hasil estimasi.

No	Nama Perangkat Lunak	Fungsi dalam Penelitian
19	OS Library	Digunakan untuk mengakses variabel lingkungan (environment variables) serta mengelola file dan direktori pada sistem operasi.
20	Python-dotenv	Digunakan untuk memuat variabel lingkungan dari file .env (seperti Groq API key) ke dalam sistem agar dapat diakses oleh aplikasi.
21	Groq API	Digunakan untuk mengakses model bahasa besar (LLM) dalam proses generation pada sistem RAG
22	LLaMa 3.1 8B	Digunakan sebagai model bahasa besar yang diakses melalui Groq API untuk menghasilkan keluaran prediksi beban tugas
23	LangChain	Digunakan untuk mengelola alur prompt, integrasi model bahasa besar, serta penggabungan proses retrieval dan generation pada RAG
24	JSON Library	Digunakan untuk melakukan parsing hasil keluaran sistem dalam format JSON menjadi struktur data yang dapat diproses lebih lanjut

2.4 Studi Kasus Penelitian

Penelitian ini menggunakan tugas akademik pada Program Studi Sistem Informasi sebagai studi kasus. Pemilihan studi kasus ini dilakukan untuk membatasi ruang lingkup penelitian agar proses pengembangan dan evaluasi sistem dapat dilakukan secara lebih terfokus. Program Studi Sistem Informasi dipilih karena karakteristik tugas akademiknya memiliki tingkat kompleksitas yang beragam, mulai dari tugas yang bersifat konseptual seperti analisis dan kajian literatur, hingga tugas yang bersifat praktis seperti perancangan sistem, pengembangan aplikasi, implementasi teknologi, dan proyek berbasis kelompok. Variasi karakteristik tersebut menjadikan tugas akademik Sistem Informasi sesuai untuk menguji kemampuan pendekatan *Retrieval-Augmented Generation* (RAG) dalam menghasilkan estimasi beban tugas berdasarkan referensi historis yang relevan. Dengan pembatasan pada satu program studi, data historis yang digunakan juga menjadi lebih homogen sehingga proses retrieval dan estimasi dapat dilakukan dalam konteks akademik yang lebih spesifik dan konsisten.

2.5 Dataset Penelitian

2.5.1 Sumber Data

Sumber data dalam penelitian ini terdiri atas dua jenis, yaitu data deskripsi tugas akademik nyata dan dataset sintesis dengan perbandingan 15 : 85. Proporsi dataset

sintetis yang lebih besar dipilih karena keterbatasan jumlah data nyata yang tersedia dalam arsip terdokumentasi, serta kebutuhan akan jumlah data yang memadai untuk mendukung proses pembelajaran dan evaluasi model secara optimal. Selain itu, penggunaan data sintetis memungkinkan peneliti memperluas variasi tingkat kompleksitas tugas secara lebih terkontrol dan sistematis, sehingga distribusi karakteristik tugas menjadi lebih beragam.

a. Data Historis

Data historis dalam penelitian ini berupa deskripsi tugas akademik nyata yang diperoleh dari riwayat tugas mahasiswa Program Studi Sistem Informasi. Data tersebut bersumber dari arsip penugasan, file tugas mahasiswa, serta deskripsi instruksi yang disampaikan oleh dosen melalui media komunikasi kelas seperti Learning Management System (LMS) dan platform komunikasi daring.

Data ini mencerminkan karakteristik umum tugas perkuliahan yang meliputi instruksi pengerjaan, ketentuan teknis, bentuk keluaran yang diminta seperti laporan, presentasi, maupun proyek sistem, serta batas waktu pengumpulan. Data historis digunakan sebagai representasi kondisi riil di lingkungan akademik dan menjadi acuan utama dalam memahami struktur serta tingkat kompleksitas tugas yang diberikan kepada mahasiswa.

Sementara itu, estimasi lama pengerjaan dan tingkat kesulitan diperoleh melalui bantuan model AI yang menganalisis deskripsi tugas akademik berdasarkan kompleksitas instruksi, jumlah pekerjaan yang harus diselesaikan, serta jenis luaran yang diminta. Hasil estimasi tersebut kemudian diverifikasi dan disesuaikan kembali oleh peneliti untuk memastikan kesesuaian dengan konteks akademik dan tingkat kesulitan tugas secara realistis. Pendekatan ini digunakan karena tidak tersedia data waktu pengerjaan aktual mahasiswa, sehingga diperlukan mekanisme estimasi semi-otomatis yang tetap mempertimbangkan validasi manusia.

b. Data Sintetis

Data sintetis dalam penelitian ini dikembangkan menggunakan bantuan sistem kecerdasan buatan generatif yang dirancang berdasarkan karakteristik data historis yang dimiliki dan pola umum penugasan di lingkungan perguruan tinggi. Proses pengembangan data sintetis dilakukan dengan mempertimbangkan variasi tingkat kompleksitas tugas, jenis instruksi, serta bentuk keluaran yang relevan dengan kondisi nyata.

Selanjutnya, dataset sintetis melalui tahapan seleksi, penyesuaian, dan verifikasi ulang untuk memastikan kesesuaian dengan karakteristik data historis. Penggunaan data sintetis bertujuan untuk memperbanyak jumlah data, meningkatkan keberagaman variasi tugas, serta mendukung proses pengujian sistem secara lebih sistematis dan terkontrol. Dengan demikian, model yang dikembangkan dapat memiliki kemampuan generalisasi yang lebih baik terhadap berbagai jenis tugas akademik.

2.5.2 Pengumpulan Data

Data tugas akademik nyata dan data sintetis dikumpulkan dan diintegrasikan menjadi satu dataset yang digunakan dalam penelitian. Proses pengumpulan data dilakukan melalui pengisian formulir daring menggunakan Google Form. Google Form dipilih karena menyediakan fitur pembatasan input sesuai dengan spesifikasi data yang dibutuhkan, sehingga dapat menjaga konsistensi struktur data yang dikumpulkan. Selain itu, fitur pemantauan secara real-time Google form dan integrasi dengan Google Spreadsheet yang memudahkan peneliti dalam mengontrol proses pengumpulan dan perkembangan dataset, serta menjaga data-data dalam dataset untuk tetap balance.

2.5.3 Deskripsi Dataset

Dataset dalam penelitian ini terdiri dari 5 kolom dan 278 baris data. Dataset disusun dari tugas akademik pada berbagai mata kuliah di Program Studi Sistem Informasi Universitas Hasanuddin. Mata kuliah tersebut mencakup berbagai bidang keilmuan, seperti pemrograman dan pengembangan aplikasi, basis data, data sains dan kecerdasan buatan, keamanan informasi, jaringan komputer, sistem operasi, rekayasa perangkat lunak, desain dan interaksi pengguna, sistem informasi dan manajemen, big data, transformasi digital, serta teknologi pendukung seperti Business Intelligence, Sistem Informasi Geografis, Fundamental ERP, dan Augmented and Virtual Reality.

Untuk melihat distribusi sumber tugas berdasarkan mata kuliah, dilakukan rekapitulasi jumlah data pada setiap mata kuliah sebagaimana ditunjukkan pada **Tabel 5**.

Tabel 5 Distribusi jumlah data setiap mata kuliah pada dataset

No	Mata Kuliah (Disesuaikan)	Jumlah
1	Keamanan Sistem Informasi	17
2	Big Data	16
3	Pengantar Pemrograman	16
4	Pemrograman Web Lanjutan	15
5	Sistem Basis Data	15
6	UI/UX	14
7	Rekayasa Perangkat Lunak	14
8	Transformasi Digital	12
9	Pemrograman Web	11
10	Pemrograman Mobile	11
11	Pengantar Data Sains	11
12	Pengantar Sistem Informasi	10
13	Business Intelligence	10
14	Rekayasa Sistem Informasi Cerdas	9

No	Mata Kuliah (Disesuaikan)	Jumlah
15	Machine Learning	8
16	Pengantar Data Mining	8
17	Pemrograman Berorientasi Objek	7
18	Visualisasi Informasi	7
19	Teknologi Jaringan dan Web	7
20	Sistem Operasi	6
21	Software Testing and Quality Assurance	6
22	Augmented and Virtual Reality	6
23	Desain dan Analisis Algoritma	6
24	Deep Learning	5
25	Basis Data Lanjutan	5
26	Pengantar Bisnis dan Manajemen	5
27	Sistem Informasi Geografis	5
28	Manajemen Proyek Sistem Informasi	5
29	Fundamental ERP	5
30	Logika Komputer	5

Berdasarkan **Tabel 5**, dataset mencakup berbagai jenis mata kuliah dengan jumlah data yang berbeda pada setiap kategori. Perbedaan jumlah data terjadi karena menyesuaikan ketersediaan data historis dan proses pengembangan data sintesis. Meskipun jumlah data antar mata kuliah tidak sepenuhnya sama, keberagaman sumber tugas tetap dijaga untuk memberikan variasi karakteristik tugas akademik.

Keragaman mata kuliah tersebut memberikan variasi aktivitas pengerjaan, seperti implementasi perangkat lunak, pengolahan data, analisis sistem, perancangan aplikasi, pengujian sistem, serta aktivitas berbasis desain. Variasi karakteristik tugas ini mendukung proses retrieval pada pendekatan *Retrieval-Augmented Generation* (RAG), karena sistem melakukan pencarian referensi historis berdasarkan kemiripan karakteristik tugas, bukan hanya berdasarkan kategori mata kuliah.

Setiap data terdiri atas tiga jenis data, yaitu timestamp dengan tipe data waktu (*datetime*), *task_teks* berupa teks tidak terstruktur, serta atribut numerik. Data kualitatif berbentuk deskripsi tugas akademik (*task_teks*) yang memuat instruksi pengerjaan, ketentuan teknis, jumlah luaran yang diminta, serta kriteria atau batasan tertentu dalam penyelesaian tugas. Teks tersebut bersifat *unstructured teks* sehingga dianalisis menggunakan pendekatan *Natural Language Processing* (NLP) untuk direpresentasikan ke dalam bentuk vektor embedding sebagai representasi numerik dari kompleksitas dan karakteristik tugas.

Adapun data kuantitatif yang digunakan sebagai variabel target dalam proses estimasi meliputi estimasi tenggat waktu dalam satuan hari (*deadline_days*), estimasi lama pengerjaan dalam satuan jam (*est_time_hours*), serta tingkat kesulitan tugas dalam skala 1–5 (*difficulty*). Kombinasi antara data teks dan data numerik ini memungkinkan sistem memodelkan hubungan antara kompleksitas deskripsi tugas

dan estimasi beban akademik secara terukur dan sistematis. Secara struktural, dataset memiliki beberapa atribut utama sebagaimana ditunjukkan pada **Tabel 6**.

Tabel 6. Struktur dan Atribut Dataset

No	Nama Kolom	Tipe Data	Deskripsi
1	Timestamp	Datetime	Waktu pencatatan atau pengisian data
2	task_teks	Teks (Unstructured)	Deskripsi tugas akademik
3	deadline_days	Integer	Estimasi tenggat waktu dalam satuan hari
4	est_time_hours	Float/Integer	Estimasi lama pengerjaan dalam satuan jam
5	difficulty	Integer (1–5)	Tingkat kesulitan tugas

2.5.4 Proses Validasi dan Pembersihan Data

Proses validasi dan pembersihan data dilakukan untuk memastikan kualitas dataset sebelum digunakan dalam proses indexing, retrieval, dan evaluasi sistem. Tahapan yang dilakukan di google colaboratory ini bertujuan untuk menghilangkan inkonsistensi, nilai tidak valid, serta atribut yang tidak relevan terhadap mekanisme kerja sistem. Tahapan yang dilakukan meliputi :

1. Penghapusan atribut tidak relevan
Atribut *Timestamp* dihapus karena hanya berfungsi sebagai metadata pencatatan waktu yang muncul saat pembuatan data di google form dan tidak memiliki kontribusi terhadap proses embedding, retrieval, maupun estimasi beban tugas. Penghapusan ini dilakukan untuk mengurangi redundansi informasi dan memastikan dataset hanya memuat atribut yang relevan terhadap tujuan penelitian.
2. Penghapusan data duplikat
Data dengan isi identik dihapus untuk menghindari bias distribusi pada proses retrieval.
3. Pemeriksaan nilai kosong atau tidak valid
Baris data yang memiliki nilai kosong atau format tidak sesuai diperbaiki atau dihapus sesuai kebutuhan.
4. Normalisasi format numerik
Atribut *deadline_days* dan *est_time_hours* dipastikan dalam format numerik yang konsisten agar dapat digunakan dalam proses evaluasi kuantitatif.
5. Pemeriksaan konsistensi skala kesulitan
Atribut *difficulty* divalidasi agar berada dalam rentang 1–5 sesuai dengan skala yang ditetapkan.

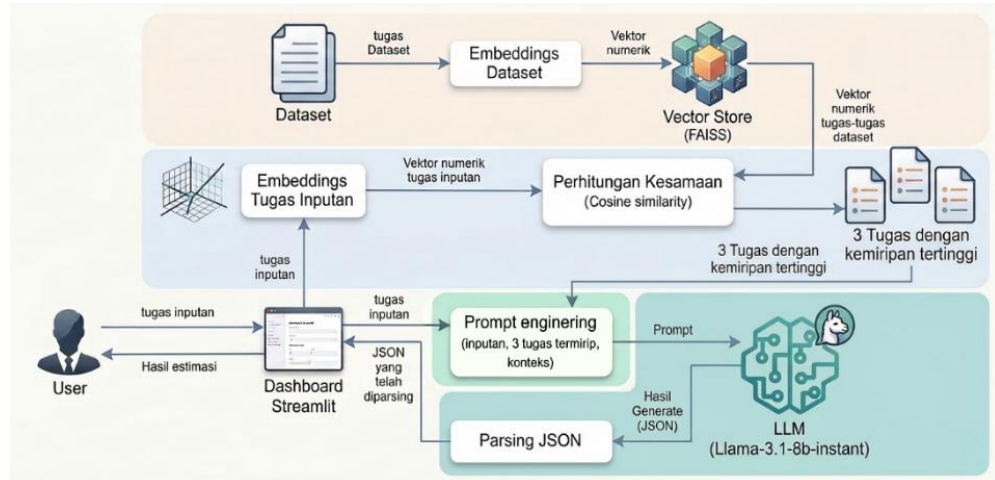
Setelah seluruh tahapan tersebut dilakukan, jumlah dataset berkurang dari 278 menjadi 277 baris data. Dataset yang telah melalui tahap validasi dan perbaikan inilah yang selanjutnya valid dan siap digunakan dalam perancangan sistem.

2.6 Perancangan Sistem

Perancangan arsitektur sistem bertujuan untuk menggambarkan struktur konseptual dan alur kerja sistem estimasi beban tugas berbasis *Retrieval-Augmented Generation* (RAG). Arsitektur ini dirancang untuk mengintegrasikan mekanisme pencarian informasi berbasis kesamaan semantik (*semantic retrieval*) dengan model bahasa generatif guna menghasilkan estimasi beban tugas yang lebih relevan dan kontekstual.

Pendekatan RAG dipilih karena mampu menggabungkan dua komponen utama, yaitu *Retrieval component* untuk mengambil referensi tugas historis yang relevan dan *Generative component* untuk melakukan analisis dan menghasilkan estimasi berdasarkan konteks yang diberikan. Dengan pendekatan ini, sistem tidak hanya mengandalkan pengetahuan umum dari model bahasa besar (*Large Language Model*), tetapi juga memanfaatkan data historis lokal Program Studi Sistem Informasi sebagai referensi tambahan dalam proses estimasi.

2.6.1 Pipeline Sistem



Gambar 4. Pipeline Sistem RAG Estimasi Beban Tugas Akademik

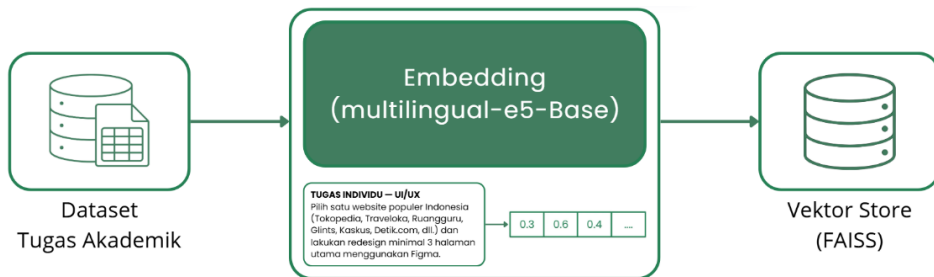
Pipeline sistem terdiri dari proses offline yang merupakan proses persiapan dan proses online yang dimulai saat tugas inputan dimasukkan. Secara keseluruhan arsitektur pembangun sistem terdiri atas beberapa komponen utama sebagai berikut:

1. **Input Layer**
Komponen ini menerima masukan berupa deskripsi tugas akademik yang diberikan oleh dosen. Teks ini menjadi sumber utama analisis beban kerja.
2. **Embedding Model**
Deskripsi tugas yang diterima akan dikonversi menjadi representasi vektor numerik menggunakan model SentenceTransformer (multilingual-e5-base). Proses embedding ini bertujuan untuk merepresentasikan makna semantik teks dalam bentuk vektor dan memungkinkan pengukuran kesamaan antar tugas.
3. **Vector Store (FAISS)**
Seluruh data tugas historis yang telah diubah menjadi embedding disimpan dalam basis data vektor menggunakan FAISS (*Facebook AI Similarity Search*). FAISS berfungsi untuk menyimpan embedding secara efisien dan melakukan pencarian kemiripan *Cosine similarity*.
4. **Retrieval 3 Tugas dengan Kemiripan Tertinggi**
Sistem mengambil 3 tugas dengan tingkat kemiripan tertinggi terhadap input pengguna. mekanisme ini memungkinkan sistem memperoleh referensi tugas serupa atau informasi historis sebagai pembanding kompleksitas.
5. **Prompt Builder**
Hasil retrieval kemudian digunakan untuk membangun prompt yang akan diberikan kepada model generatif. Prompt dirancang untuk memberikan konteks tugas historis, menginstruksikan model melakukan analisis kompleksitas, dan menghasilkan estimasi dalam format JSON terstruktur.
6. **Generative Model (LLaMa 3.1 8B)**
Model generatif yang digunakan adalah model LLaMa 3.1 8B dari Meta dengan ID LLaMa-3.1-8b-instant. Model ini menerima prompt yang telah diperkaya dengan konteks retrieval, kemudian melakukan analisis terhadap jenis tugas, komponen kerja, kebutuhan teknis (database, ML, UI/UX, dsb), estimasi waktu pengerjaan, tingkat kesulitan, sehingga Model menghasilkan keluaran berupa estimasi terstruktur dalam format JSON.
7. **Keluaran Parser**
Keluaran yang dihasilkan oleh model generatif diproses menggunakan parser untuk memastikan format JSON valid. Komponen ini bertujuan untuk menghindari kesalahan parsing dan mengambil nilai estimasi secara akurat.
8. **Dashboard (Streamlit)**
Seluruh proses diintegrasikan dalam dashboard berbasis Streamlit, yang menyediakan antarmuka pengguna untuk memasukkan deskripsi tugas, menampilkan hasil estimasi (deadline, estimasi jam, difficulty), reasoning estimasi, dan referensi tugas serupa.

2.6.2 Indexing

Indexing merupakan proses persiapan yang dilaksanakan secara offline saat inisialisasi sistem yang bertujuan untuk menyiapkan vektor numerik dataset sebagai

pembandingan pada tahap online atau Retrieval. Berikut alur konseptual yang terjadi proses indexing.



Gambar 5. Tahapan dalam proses Indexing

2.6.2.1 Embedding Dataset

Tahap embedding merupakan proses transformasi teks deskripsi tugas akademik menjadi representasi numerik dalam bentuk vektor sehingga dapat diproses secara matematis pada tahap retrieval. Dalam penelitian ini, model embedding yang digunakan adalah *multilingual-e5-base* yang tersedia melalui pustaka *Sentence Transformers* pada platform *Hugging Face*. Model ini menghasilkan representasi vektor berdimensi 768 dan telah dilatih untuk tugas *semantic textual similarity*, sehingga mampu merepresentasikan makna kalimat dalam ruang vektor kontinu. Tujuan utama dari tahap embedding adalah mengubah data teks yang bersifat tidak terstruktur menjadi bentuk numerik yang dapat dihitung kemiripannya. Dengan pendekatan ini, sistem tidak hanya membandingkan kata secara literal, tetapi juga mempertimbangkan kedekatan makna antar kalimat. Representasi vektor yang dihasilkan memungkinkan sistem menghitung jarak atau tingkat kemiripan antara deskripsi tugas baru dan data historis secara lebih akurat.

Pada proses embedding offline, seluruh deskripsi tugas historis dalam dataset yang telah dibersihkan (*dataset_clean.csv*) diproses menggunakan model *multilingual-e5-base*. Kolom *task_teks* diekstraksi sebagai input teks, kemudian setiap teks dikonversi menjadi satu vektor berdimensi 768 menggunakan metode encoding dengan ukuran *batch* 16 untuk menjaga efisiensi komputasi. Apabila jumlah data historis adalah n , maka hasil proses embedding membentuk matriks berdimensi $n \times 768$. Matriks tersebut kemudian disimpan dalam file *embeddings.npy* agar tidak perlu dihitung ulang setiap kali sistem dijalankan, sehingga meningkatkan efisiensi pada tahap retrieval.

Model *multilingual-e5-base* dipilih sebagai model embedding karena mendukung representasi semantik multibahasa, termasuk bahasa Indonesia dengan performa yang baik sehingga sangat cocok pada pendekatan *Retrieval-Augmented Generation* (RAG).

Dengan pendekatan ini, deskripsi tugas yang memiliki makna serupa akan memiliki jarak vektor yang lebih dekat dalam ruang embedding, meskipun tidak menggunakan kata-kata yang identik secara eksplisit. Dengan demikian, tahap embedding menjadi pondasi utama dalam sistem retrieval berbasis semantik yang dikembangkan dalam penelitian ini, karena seluruh proses pencarian konteks bergantung pada kualitas representasi vektor yang dihasilkan pada tahap ini.

2.6.2.2 Penyimpanan ke vektor Faiss

Setelah seluruh deskripsi tugas historis dikonversi menjadi representasi vektor pada tahap embedding, langkah berikutnya adalah menyimpan vektor-vektor tersebut ke dalam struktur indeks untuk mendukung proses pencarian semantik. Dalam penelitian ini, penyimpanan dan pengelolaan indeks vektor dilakukan menggunakan pustaka FAISS (*Facebook AI Similarity Search*) yang dikembangkan oleh Meta AI.

FAISS digunakan sebagai vector index yang memungkinkan proses pencarian nearest neighbor secara efisien dalam ruang berdimensi tinggi. Mengingat setiap deskripsi tugas direpresentasikan sebagai vektor berdimensi 768, diperlukan mekanisme pencarian yang optimal agar sistem tidak perlu menghitung jarak secara manual terhadap seluruh data setiap kali menerima query baru.

Pada tahap inisialisasi sistem, embedding seluruh deskripsi tugas historis yang telah tersimpan dalam bentuk matriks dimuat ke dalam memori dan dikonversi ke format numerik *float32* agar kompatibel dengan FAISS. FAISS digunakan karena mampu melakukan pencarian kemiripan vektor secara efisien pada data embedding berdimensi tinggi serta mendukung perhitungan *cosine similarity* untuk proses retrieval berbasis semantik dalam pendekatan *Retrieval-Augmented Generation* (RAG). Dimensi vektor ditentukan berdasarkan jumlah kolom embedding, kemudian dibentuk struktur indeks menggunakan tipe *IndexFlatIP* yang digunakan untuk perhitungan *cosine similarity*. Sebelum dimasukkan ke dalam indeks, seluruh vektor embedding dinormalisasi terlebih dahulu menggunakan normalisasi L2 agar perhitungan *inner product* pada FAISS setara dengan *cosine similarity*. Seluruh vektor kemudian dimasukkan ke dalam indeks sehingga setiap data historis direpresentasikan sebagai titik dalam ruang vektor berdimensi 768.

Setelah proses penambahan vektor selesai, indeks disimpan dalam file `semantic_index.faiss` agar dapat digunakan kembali tanpa perlu membangun ulang struktur indeks. Selain itu, metadata dataset disimpan secara terpisah dalam file `task_meta.pkl` untuk menjaga keterkaitan antara vektor embedding dan informasi asli seperti deskripsi tugas dan label estimasi. Ketika sistem dijalankan, indeks dan metadata ini dimuat kembali ke memori sebagai dasar proses retrieval.

Proses pembangunan dan penyimpanan indeks dilakukan secara offline dan hanya diperbarui apabila terdapat penambahan atau perubahan pada data historis. Dengan demikian, tahap ini memastikan bahwa sistem memiliki struktur pencarian yang efisien dan siap digunakan pada saat menerima query pengguna dalam arsitektur *Retrieval-Augmented Generation* (RAG).

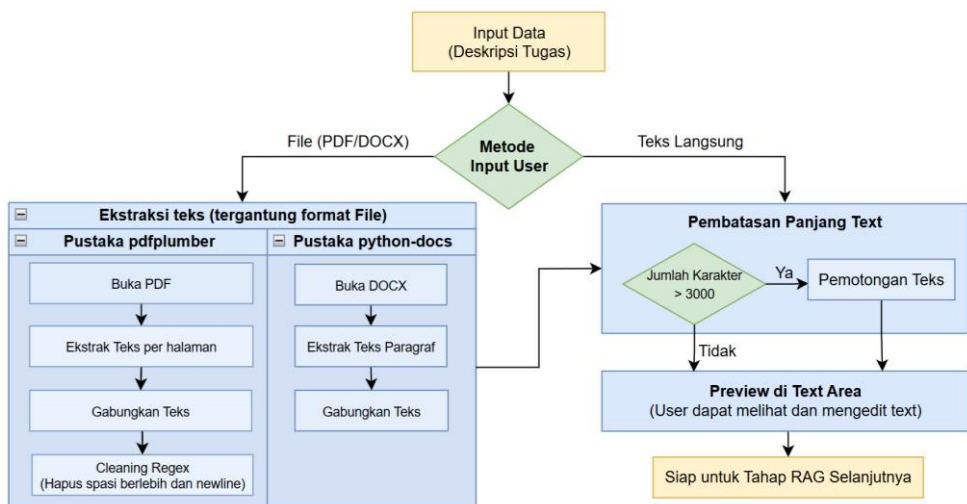
2.6.3 Retrieval

Retrieval merupakan proses online yang terjadi saat sistem menerima inputan dari pengguna yang selanjutnya akan dibandingkan dengan index yang telah dibangun pada tahap *indexing*. Tahap retrieval bertujuan untuk mencari dan mengambil konteks yang paling relevan dari vector store berdasarkan dengan inputan yang dimasukkan pengguna.

Tahapan retrieval dalam penelitian ini meliputi proses input data oleh pengguna, embedding inputan, perhitungan kemiripan, dan pengambilan Tugas dengan kemiripan tertinggi.

2.6.3.1 Penginputan Data

Input yang digunakan dalam penelitian ini berupa teks deskripsi tugas yang dapat dimasukkan secara langsung oleh pengguna maupun melalui file dalam format PDF dan DOCX. Proses dimulai ketika pengguna memasukkan teks ke dalam kolom input atau menggunakan fitur upload untuk mengunggah file. Setelah file diunggah, sistem akan mengekstrak isi dokumen dan menampilkannya kembali pada kolom input sebagai preview, sehingga pengguna dapat melakukan pengecekan atau pengeditan sebelum melanjutkan ke tahap berikutnya.



Gambar 6. Alur Pemrosesan Inputan

Proses ekstraksi teks dari file PDF dilakukan menggunakan pustaka pdfplumber. Pustaka ini memungkinkan sistem untuk membaca isi dokumen PDF per halaman dan mengekstrak teks secara langsung. Setiap halaman diproses secara berurutan, kemudian hasil teks digabungkan menjadi satu kesatuan. Setelah itu, dilakukan pembersihan teks menggunakan regular expression untuk menghapus

karakter yang tidak diperlukan, seperti spasi berlebih dan baris kosong, sehingga diperoleh teks yang lebih rapi dan konsisten untuk diproses lebih lanjut.

Sementara itu, untuk file DOCX, proses ekstraksi dilakukan menggunakan pustaka `python-docx` yang secara langsung membaca isi paragraf dalam dokumen Word. Seluruh teks dari setiap paragraf digabungkan menjadi satu string tanpa memerlukan proses pembersihan yang kompleks, karena struktur dokumen Word relatif lebih terorganisir dibandingkan PDF.

Selain proses ekstraksi, sistem juga menerapkan pembatasan panjang teks dengan menetapkan maksimum karakter sebesar 3000. Jika panjang teks hasil ekstraksi melebihi batas tersebut, maka teks akan dipotong untuk menjaga efisiensi pemrosesan serta menghindari beban berlebih pada model saat tahap embedding dan generasi.

Dengan demikian, tahap penginputan data tidak hanya berfungsi sebagai penerimaan input, tetapi juga mencakup proses normalisasi dan standarisasi teks agar sesuai untuk diproses pada tahap selanjutnya dalam *pipeline Retrieval-Augmented Generation (RAG)*.

2.6.3.2 Embedding Teks Inputan

Sama seperti embedding dataset, embedding tugas inputan atau query juga dilakukan dengan menggunakan model yang sama yaitu *multilingual-e5-base* untuk menghasilkan satu vektor embedding dengan dimensi yang sama yakni 768. Penggunaan model yang konsisten pada tahap offline dan online memastikan bahwa seluruh representasi berada dalam ruang vektor yang identik, sehingga perhitungan kemiripan dapat dilakukan secara valid tanpa terjadi perbedaan distribusi representasi.

2.6.3.3 Perhitungan Kemiripan (Similarity Measurement)

Tahap perhitungan kemiripan dilakukan setelah sistem menerima deskripsi tugas baru dari pengguna. Teks input terlebih dahulu dikonversi menjadi representasi vektor menggunakan model embedding yang sama seperti pada tahap sebelumnya. Vektor hasil embedding tersebut merepresentasikan makna deskripsi tugas dalam ruang vektor berdimensi 768. Vektor query kemudian dibandingkan dengan seluruh vektor dalam indeks FAISS yang telah dibangun pada tahap indexing.

Proses pencarian dilakukan menggunakan metode *nearest neighbor search* berbasis *cosine similarity* melalui pendekatan *inner product* yang telah disesuaikan dengan normalisasi vektor. Secara teori, nilai kemiripan yang dihasilkan *Cosine similarity* berada pada rentang -1 hingga 1, namun pada kasus embedding teks, umumnya menghasilkan vektor yang berada pada ruang positif, di mana semakin mendekati 1 menunjukkan tingkat kemiripan semantik yang semakin tinggi antara query dan data historis. Sebagai ilustrasi, jika suatu deskripsi tugas memiliki nilai kemiripan 0,89 terhadap query dan deskripsi lainnya memiliki nilai 0,82, maka

deskripsi dengan nilai 0,89 dianggap lebih relevan karena memiliki kesamaan makna yang lebih tinggi.

Pemilihan *cosine similarity* dalam penelitian ini didasarkan pada kemampuannya dalam mengukur kesamaan arah antar vektor tanpa dipengaruhi oleh panjang atau magnitudo vektor, sehingga lebih sesuai untuk membandingkan representasi teks berbasis embedding. Selain itu, metode ini umum digunakan dalam *semantic retrieval* karena mampu menangkap kedekatan makna secara lebih representatif, terutama pada data teks dengan variasi panjang dan kompleksitas yang berbeda. Penggunaan *cosine similarity* yang diintegrasikan dengan FAISS juga memungkinkan proses pencarian tetap efisien, stabil, serta mudah diimplementasikan dalam arsitektur Retrieval-Augmented Generation (RAG).

Hasil dari tahap ini berupa sejumlah data historis dengan nilai kemiripan tertinggi terhadap query pengguna (*nearest neighbors*). Data tersebut kemudian digunakan sebagai konteks tambahan pada tahap generasi estimasi oleh model bahasa besar (LLM).

2.6.3.4 Pengambilan Tugas dengan Similarity Tertinggi

Tahap pengambilan tugas dengan similarity tertinggi dilakukan setelah sistem menghitung nilai kemiripan antara vektor query dan seluruh vektor deskripsi tugas historis yang tersimpan dalam indeks FAISS. Proses ini bertujuan untuk memilih sejumlah data historis yang paling relevan berdasarkan nilai *cosine similarity* tertinggi terhadap query pengguna. Pencarian dilakukan menggunakan fungsi *search()* pada indeks FAISS berbasis *inner product* yang telah disesuaikan dengan *cosine similarity*, dengan keluaran berupa nilai kemiripan (*similarity scores*) dan indeks posisi data (*indices*) dalam dataset. Nilai kemiripan menunjukkan tingkat kedekatan semantik antara query dan data historis, sedangkan indeks digunakan untuk mengambil metadata terkait dari struktur penyimpanan terpisah. Sistem hanya memproses indeks yang valid (tidak bernilai -1) untuk memastikan konsistensi data yang digunakan sebagai konteks.

Dalam penelitian ini, nilai tugas dengan similarity tertinggi ditetapkan sebanyak 3, yang berarti sistem mengambil tiga deskripsi tugas historis dengan nilai kemiripan tertinggi. Pemilihan jumlah ini didasarkan pada pertimbangan keseimbangan antara relevansi dan kelengkapan konteks, efisiensi dalam penyusunan prompt, serta kemampuan model dalam melakukan generalisasi tanpa terpengaruh oleh terlalu banyak informasi yang berpotensi menimbulkan *noise*.

Dalam arsitektur Retrieval-Augmented Generation (RAG), tahap ini berperan penting karena data historis terpilih akan digunakan sebagai konteks tambahan dalam proses generasi oleh Large Language Model (LLM). Ketiga deskripsi tugas tersebut dimasukkan ke dalam prompt sebagai referensi historis untuk membantu model dalam menghasilkan estimasi yang lebih akurat dan kontekstual. Dengan demikian, tahap pengambilan tugas dengan similarity tertinggi berfungsi sebagai mekanisme seleksi informasi yang secara langsung mempengaruhi kualitas hasil akhir sistem.

Dengan demikian, tahap pengambilan tugas dengan similarity tertinggi berfungsi sebagai mekanisme penyaring informasi yang menentukan kualitas konteks pada tahap generasi. Ketepatan dalam pemilihan jumlah dan relevansi data historis secara langsung mempengaruhi kualitas estimasi yang dihasilkan dalam sistem RAG yang dikembangkan.

2.6.4 Generasi Estimasi

Metode generasi estimasi merupakan tahap utama dalam sistem yang bertugas menghasilkan prediksi beban tugas mahasiswa berdasarkan deskripsi tugas yang diberikan oleh dosen. Tahap ini memanfaatkan pendekatan Retrieval-Augmented Generation (RAG), yaitu mengombinasikan hasil retrieval deskripsi tugas historis dengan kemampuan generatif Large Language Model (LLM). Dengan pendekatan ini, estimasi yang dihasilkan tidak hanya berbasis pemahaman umum model terhadap bahasa, tetapi juga dikalibrasi menggunakan konteks tugas-tugas sebelumnya yang relevan dalam lingkungan Program Studi Sistem Informasi.

Secara operasional, setelah sistem memperoleh tiga tugas historis paling relevan melalui mekanisme semantic retrieval, data tersebut dimasukkan ke dalam prompt sebagai referensi kontekstual. LLM kemudian menganalisis deskripsi tugas baru dengan mempertimbangkan referensi tersebut untuk menghasilkan estimasi beban tugas.

Berbeda dengan pendekatan prediksi nilai tunggal, penelitian ini menggunakan pendekatan estimasi berbentuk rentang (*range estimation*) pada variabel deadline dan lama pengerjaan. Pendekatan ini dipilih karena beban pengerjaan tugas akademik pada praktiknya memiliki variasi yang dipengaruhi oleh kemampuan mahasiswa, pengalaman sebelumnya, tingkat pemahaman materi, serta kondisi pelaksanaan tugas. Oleh karena itu, sistem menghasilkan rentang estimasi yang terdiri atas batas minimum dan batas maksimum untuk variabel deadline dan lama pengerjaan. Sementara itu, tingkat kesulitan (*difficulty*) disajikan dalam bentuk nilai tunggal pada skala 1–5 sebagai representasi tingkat kesulitan umum dari tugas yang dianalisis.

Keluaran yang dihasilkan sistem disajikan dalam format JSON terstruktur yang memuat rentang estimasi tenggat waktu (*deadline_min_days* dan *deadline_max_days*), rentang estimasi lama pengerjaan (*est_time_min_hours* dan *est_time_max_hours*), tingkat kesulitan tugas (*difficulty*), serta *reasoning* yang menjelaskan dasar pemberian estimasi berdasarkan referensi historis yang diperoleh pada tahap retrieval.

2.6.4.1 Konfigurasi Large Language Model (LLM)

Model generatif yang digunakan dalam penelitian ini adalah LLaMa 3.1 8B, yang dikembangkan oleh Meta dan diakses melalui API dari platform Groq. Model ini merupakan bagian dari keluarga LLaMa (*Large Language Model Meta AI*) yang dirancang untuk menghasilkan teks secara kontekstual dengan memanfaatkan

arsitektur transformer. Dengan jumlah parameter sekitar 8 miliar, model ini memiliki kemampuan yang baik dalam memahami dan menghasilkan bahasa alami secara efisien.

Penggunaan API Groq memungkinkan integrasi model ke dalam sistem berbasis Python secara langsung melalui mekanisme autentikasi API key. Pendekatan ini memungkinkan proses generasi dilakukan tanpa memerlukan infrastruktur komputasi lokal, sehingga lebih praktis dan efisien dalam pengembangan sistem berbasis Streamlit. Selain itu, penggunaan API memastikan konsistensi akses model selama eksperimen serta mendukung reproduibilitas penelitian.

Pemilihan LLaMa 3.1 8B didasarkan pada pertimbangan keseimbangan antara kecepatan respons dan kemampuan pemahaman konteks. Model ini dirancang untuk memberikan latensi rendah (*low latency*), sehingga sesuai untuk sistem berbasis Retrieval-Augmented Generation (RAG) yang membutuhkan proses retrieval dan generasi secara interaktif. Selain itu, model ini memiliki kemampuan *instruction-following* yang cukup stabil dalam menghasilkan keluaran terstruktur, sehingga mendukung kebutuhan sistem dalam menghasilkan estimasi berbasis format tertentu.

Dibandingkan dengan model berukuran lebih besar, LLaMa 3.1 8B menawarkan efisiensi komputasi yang lebih baik tanpa mengorbankan kualitas pemahaman secara signifikan, khususnya pada analisis teks dengan kompleksitas menengah seperti deskripsi tugas akademik. Oleh karena itu, model ini dinilai sesuai sebagai komponen generatif dalam penelitian ini.

Dalam sistem yang dikembangkan, model bahasa generatif memiliki beberapa peran utama. Pertama, model menganalisis karakteristik dan kompleksitas deskripsi tugas akademik yang diberikan pengguna. Kedua, model menginterpretasikan konteks yang diperoleh dari referensi historis hasil proses *retrieval* sebagai bahan pembandingan untuk menilai apakah tugas baru memiliki beban yang lebih ringan, setara, atau lebih berat dibandingkan tugas sebelumnya. Ketiga, berdasarkan analisis tugas dan konteks historis tersebut, model menghasilkan estimasi beban tugas yang mencakup rentang tenggat waktu (*deadline*), estimasi lama pengerjaan, tingkat kesulitan tugas (*difficulty*), serta reasoning yang menjelaskan dasar pemberian estimasi. Seluruh hasil tersebut kemudian disajikan dalam format JSON terstruktur agar dapat diproses dan ditampilkan oleh sistem secara konsisten.

Pada sistem, parameter temperature diatur sebesar 0 untuk memastikan keluaran model bersifat deterministik dan konsisten. Pengaturan ini bertujuan untuk meminimalkan variasi respons yang dihasilkan model terhadap input yang sama, sehingga mendukung stabilitas format keluaran JSON dan meningkatkan keandalan dalam proses evaluasi. Dengan demikian, hasil estimasi yang dihasilkan lebih dapat direproduksi dalam skenario eksperimen.

Dengan demikian, LLM dalam penelitian ini tidak hanya berfungsi sebagai generator teks, tetapi sebagai komponen inferensi yang terarah dan dikendalikan oleh konteks retrieval dalam arsitektur Retrieval-Augmented Generation (RAG).

Integrasi melalui API Groq memastikan proses generasi berjalan secara konsisten, efisien, dan dapat direplikasi dalam lingkungan eksperimen yang sama.

2.6.4.2 Prompt Engineering

Untuk memastikan model menghasilkan estimasi yang konsisten, terstruktur, dan berbasis kompleksitas tugas yang aktual, penelitian ini menerapkan pendekatan prompt engineering terkontrol. Prompt dirancang untuk menggabungkan konteks hasil retrieval dengan deskripsi tugas baru serta memberikan instruksi keluaran dalam format JSON terstandarisasi. Struktur prompt yang digunakan ditunjukkan pada **Tabel 7**.

Tabel 7. Struktur prompt sistem

Komponen Prompt	Fungsi	Instruksi dalam Prompt
Referensi Historis	Menyediakan konteks berupa tugas-tugas akademik yang memiliki kemiripan semantik tertinggi dengan tugas baru dan digunakan sebagai bahan perbandingan beban tugas.	Referensi historis mahasiswa Program Studi Sistem Informasi angkatan sebelumnya: {retrieved_tasks}
Peran Model	Menentukan perspektif analisis yang digunakan model dalam melakukan estimasi.	Anda adalah dosen Sistem Informasi yang berpengalaman dalam merancang, memberikan, dan mengevaluasi tugas akademik.
Tugas Baru	Menyediakan deskripsi tugas yang akan dianalisis oleh model.	Setelah mengajar materi yang relevan, Anda akan memberikan tugas baru berikut kepada mahasiswa: {task_teks}
Instruksi Analisis	Mengarahkan model untuk membandingkan tugas baru dengan referensi historis berdasarkan kesamaan aktivitas utama, ruang lingkup, luaran, teknologi, dan tingkat kompleksitas, kemudian menggunakan referensi yang paling relevan sebagai dasar proses estimasi tanpa melakukan perhitungan statistik sederhana.	Bandinkan tugas baru dengan setiap referensi historis berdasarkan kesamaan aktivitas utama. Pertimbangkan pula ruang lingkup, jenis luaran, teknologi yang digunakan, dan tingkat kompleksitas sebagai faktor pendukung. Gunakan referensi yang paling relevan sebagai dasar utama estimasi, bukan dengan menghitung rata-rata atau menyalin nilai estimasi dari referensi historis.

Komponen Prompt	Fungsi	Instruksi dalam Prompt
Instruksi Estimasi	Menentukan variabel yang harus dihasilkan model berdasarkan hasil analisis tugas.	Tentukan nilai yang realistis untuk <code>deadline_min_days</code> , <code>deadline_max_days</code> , <code>est_time_min_hours</code> , <code>est_time_max_hours</code> , dan <code>difficulty</code> .
Aturan Estimasi	Membatasi rentang estimasi agar tetap realistis dan tidak terlalu lebar.	Deadline dan lama pengerjaan harus terkait, Asumsi mahasiswa mengerjakan tugas 1-2 jam per hari, selisih deadline tidak boleh melebihi 7 hari, selisih estimasi waktu tidak boleh melebihi 40% dari batas minimum, dan rentang harus tetap merepresentasikan mahasiswa yang mampu menyelesaikan tugas dengan baik.
Reasoning	Meminta model menjelaskan karakteristik tugas baru, membandingkannya dengan referensi historis, serta menjelaskan alasan pemberian seluruh nilai estimasi.	Menjelaskan karakteristik tugas baru secara singkat, hubungan dengan referensi historis, tingkat kompleksitas (lebih sederhana, setara, atau lebih kompleks), serta ditulis secara koheren tanpa pengulangan antar referensi dan fokus pada aspek yang memengaruhi kompleksitas tugas.
Format Keluaran	Memastikan keluaran model memiliki struktur yang konsisten dan mudah diproses sistem.	Keluaran hanya JSON dengan atribut <code>deadline_min_days</code> , <code>deadline_max_days</code> , <code>est_time_min_hours</code> , <code>est_time_max_hours</code> , <code>difficulty</code> , dan <code>reasoning</code> .

Secara konseptual, prompt dirancang agar model berperan sebagai dosen yang melakukan analisis terhadap beban tugas akademik sebelum tugas diberikan kepada mahasiswa. Dalam proses tersebut, hasil retrieval tidak digunakan sebagai sumber jawaban yang harus ditiru, melainkan sebagai referensi historis untuk membantu proses penalaran. Model terlebih dahulu menganalisis karakteristik tugas baru, kemudian membandingkannya dengan setiap referensi historis berdasarkan kesamaan aktivitas utama yang dilakukan mahasiswa. Selain itu, model juga

mempertimbangkan kesamaan ruang lingkup, jenis luaran, teknologi yang digunakan, dan tingkat kompleksitas sebagai faktor pendukung. Selanjutnya, referensi yang paling relevan dijadikan dasar utama dalam menentukan apakah tugas baru memiliki beban yang lebih ringan, setara, atau lebih berat dibandingkan tugas sebelumnya. Hasil analisis tersebut kemudian digunakan sebagai dasar dalam menentukan rentang estimasi deadline, rentang estimasi waktu pengerjaan, dan tingkat kesulitan tugas.

Selain itu, prompt secara eksplisit menginstruksikan model untuk tidak menghasilkan estimasi berdasarkan rata-rata atau perhitungan statistik sederhana dari data historis. Sebaliknya, model diminta melakukan analisis kontekstual terhadap tugas baru dan menentukan rentang estimasi yang realistis berdasarkan tingkat kompleksitas yang teridentifikasi. Pendekatan ini bertujuan agar estimasi yang dihasilkan lebih adaptif terhadap karakteristik masing-masing tugas dan tidak sekadar mereplikasi nilai yang terdapat pada referensi historis.

Prompt juga dilengkapi dengan beberapa aturan tambahan untuk menjaga konsistensi dan kewajaran hasil estimasi. Deadline minimum harus mempertimbangkan estimasi lama pengerjaan dengan asumsi mahasiswa mengerjakan tugas sekitar 1–2 jam per hari sebagai pedoman kewajaran, sehingga estimasi deadline dan lama pengerjaan tetap memiliki keterkaitan yang logis. Selain itu, selisih antara deadline minimum dan maksimum dibatasi maksimal tujuh hari, sedangkan selisih antara estimasi waktu minimum dan maksimum dibatasi maksimal 40% dari nilai minimum. Aturan-aturan tersebut diterapkan karena selama proses pengembangan sistem model bahasa cenderung menghasilkan rentang estimasi yang terlalu lebar atau menghasilkan deadline yang tidak sebanding dengan estimasi lama pengerjaan apabila tidak diberikan batasan yang jelas. Pembatasan ini bertujuan agar hasil estimasi tetap realistis, konsisten, informatif, dan mampu merepresentasikan variasi kemampuan mahasiswa yang masih berada pada tingkat yang wajar.

Keluaran sistem disajikan dalam format JSON yang memuat rentang estimasi tenggat waktu (*deadline_min_days* dan *deadline_max_days*), rentang estimasi lama pengerjaan (*est_time_min_hours* dan *est_time_max_hours*), tingkat kesulitan tugas (*difficulty*), serta *reasoning*. *Reasoning* berfungsi menjelaskan karakteristik tugas baru, hasil perbandingan dengan referensi historis yang digunakan sebagai pembandingan, dan alasan pemberian setiap nilai estimasi yang dihasilkan sistem.

Dengan demikian, strategi *prompt engineering* yang diterapkan bertujuan untuk mengurangi risiko *hallucination*, mencegah penyalinan langsung dari referensi historis, mendorong model melakukan analisis berbasis konteks, serta memastikan keluaran yang dihasilkan sesuai dengan struktur data yang dibutuhkan oleh sistem. Pendekatan ini menjadikan tahap generasi estimasi lebih terkontrol dan selaras dengan tujuan utama arsitektur *Retrieval-Augmented Generation* (RAG), yaitu menghasilkan estimasi beban tugas yang kontekstual, transparan, dan adaptif terhadap karakteristik tugas akademik pada Program Studi Sistem Informasi.

2.6.4.3 Struktur Keluaran dalam Format JSON

Model dikonfigurasi untuk menghasilkan keluaran dalam format JSON terstruktur agar mudah diproses dan diintegrasikan ke dalam sistem. Format ini dipilih karena mendukung pengolahan data secara langsung dalam pipeline aplikasi serta meminimalkan kesalahan parsing pada tahap implementasi. Selain itu, penggunaan format JSON memungkinkan setiap hasil estimasi ditampilkan secara konsisten pada dashboard tanpa memerlukan proses konversi tambahan. Struktur keluaran yang digunakan dalam penelitian ini disajikan pada **Tabel 8**.

Tabel 8. Struktur Keluaran Sistem

Atribut	Tipe Data	Deskripsi
deadline_min_days	Integer	Estimasi minimum jumlah hari yang realistis untuk penyelesaian tugas
deadline_max_days	Integer	Estimasi maksimum jumlah hari yang masih dianggap wajar untuk penyelesaian tugas
est_time_min_hours	Float	Estimasi minimum waktu pengerjaan bagi mahasiswa yang memahami materi dengan baik dan bekerja secara efisien
est_time_max_hours	Float	Estimasi maksimum waktu pengerjaan bagi mahasiswa dengan kemampuan rata-rata yang tetap mampu menyelesaikan tugas dengan baik
difficulty	Integer	Tingkat kesulitan tugas pada skala 1–5
reasoning	String	Penjelasan alasan estimasi yang dihasilkan model berdasarkan analisis tugas dan referensi historis

Setiap parameter memiliki fungsi yang berbeda dalam merepresentasikan beban tugas akademik. Atribut *deadline_min_days* dan *deadline_max_days* digunakan untuk menggambarkan rentang tenggat waktu yang realistis, sedangkan *est_time_min_hours* dan *est_time_max_hours* digunakan untuk merepresentasikan rentang lama pengerjaan yang mungkin dibutuhkan mahasiswa. Variabel *difficulty* digunakan untuk menunjukkan tingkat kesulitan umum tugas pada skala 1–5, sementara *reasoning* berisi penjelasan yang menjelaskan dasar pertimbangan model dalam menghasilkan estimasi tersebut.

Penggunaan pendekatan rentang (*range estimation*) pada variabel deadline dan lama pengerjaan bertujuan untuk mengakomodasi variasi kemampuan, pengalaman, dan tingkat pemahaman mahasiswa terhadap materi yang diberikan. Dengan demikian, hasil estimasi tidak hanya merepresentasikan satu nilai tunggal, tetapi memberikan gambaran yang lebih fleksibel mengenai kemungkinan beban tugas yang dapat dialami mahasiswa.

Selain memudahkan integrasi dengan dashboard Streamlit, struktur JSON ini juga mendukung proses evaluasi sistem melalui penilaian kelayakan hasil estimasi (*manual judgement*), evaluasi kualitas reasoning menggunakan pendekatan *LLM-as-a-Judge*, serta pengujian konsistensi format keluaran menggunakan *Parsing Success Rate (PSR)*. Dengan struktur yang terstandarisasi, hasil keluaran model dapat dimanfaatkan secara langsung baik untuk kebutuhan visualisasi maupun evaluasi performa sistem.

2.6.4.4 Mekanisme Parsing JSON

Meskipun model telah dikonfigurasi untuk menghasilkan keluaran dalam format JSON, terdapat kemungkinan bahwa model LLaMa 3.1 8B masih menghasilkan teks tambahan di luar struktur JSON, seperti penjelasan naratif atau format markdown (misalnya penggunaan ``json). Oleh karena itu, sistem menerapkan mekanisme parsing untuk memastikan bahwa hanya data terstruktur yang diproses lebih lanjut. Proses parsing dilakukan melalui beberapa langkah, yaitu:

1. Membersihkan tanda format seperti ``json atau backticks.
2. Menggunakan *regular expression* untuk mengekstrak blok teks berbentuk JSON.
3. Melakukan validasi struktur JSON menggunakan fungsi `json.loads()`.
4. Mengembalikan nilai `None` apabila struktur tidak valid atau terjadi kesalahan parsing.

Pada tahap implementasi, hasil keluaran model dari API Groq terlebih dahulu disimpan dalam bentuk teks, kemudian diproses menggunakan fungsi parsing tersebut. Jika proses parsing berhasil, data akan ditampilkan pada dashboard Streamlit dalam bentuk metrik terstruktur, seperti *deadline*, *difficulty*, dan estimasi waktu, serta penjelasan tambahan (*reasoning*). Sebaliknya, jika parsing gagal, sistem akan menampilkan pesan kesalahan dan data tidak digunakan dalam proses evaluasi.

Pendekatan ini bertujuan untuk menjaga stabilitas sistem, menghindari error saat runtime, serta memastikan bahwa hanya data yang valid dan terstruktur yang digunakan dalam perhitungan metrik evaluasi. Dengan demikian, tahap generasi dalam sistem tidak hanya bergantung pada kemampuan model bahasa, tetapi juga didukung oleh mekanisme kontrol struktural yang memastikan keluaran dapat diproses secara konsisten dan terintegrasi dengan baik dalam arsitektur *Retrieval-Augmented Generation (RAG)*.

2.6.5 Dashboard Streamlit

Antarmuka sistem dikembangkan menggunakan framework Streamlit untuk mengintegrasikan pengguna dengan seluruh komponen arsitektur *Retrieval-Augmented Generation (RAG)*. Streamlit dipilih karena kemudahannya bagi pengembang, memungkinkan implementasi berbasis Python secara langsung

tanpa pemisahan backend-frontend, serta seluruh pipeline pemrosesan berjalan dalam satu lingkungan eksekusi konsisten.

Dalam konteks metodologi, dashboard berfungsi sebagai lapisan orkestrasi yang menghubungkan beberapa tahapan utama sistem, yaitu:

1. Penerimaan deskripsi tugas sebagai input.
2. Transformasi teks menjadi embedding vektor.
3. Proses semantic retrieval menggunakan indeks FAISS.
4. Pembentukan prompt dengan referensi historis.
5. Generasi estimasi menggunakan model LLM.
6. Parsing dan validasi keluaran dalam format JSON.

Dengan demikian, dashboard tidak hanya berperan sebagai antarmuka visual, tetapi sebagai pengendali alur eksperimen yang memastikan setiap komponen berjalan secara terintegrasi dan berurutan.

Setelah proses parsing berhasil dilakukan, sistem mengekstraksi nilai *deadline_min_days*, *deadline_max_days*, *est_time_min_hours*, *est_time_max_hours*, *difficulty*, dan *reasoning* dari keluaran JSON yang dihasilkan model. Nilai-nilai tersebut kemudian ditampilkan pada dashboard dalam bentuk rentang estimasi tenggat waktu, rentang estimasi lama pengerjaan, tingkat kesulitan tugas, serta penjelasan yang menjelaskan dasar pemberian estimasi. Selain itu, referensi historis yang digunakan pada proses *retrieval* juga ditampilkan sebagai informasi pendukung untuk meningkatkan transparansi hasil yang diberikan sistem. Apabila proses parsing gagal dilakukan, sistem tidak menampilkan hasil estimasi dan akan memberikan notifikasi kesalahan kepada pengguna guna menjaga konsistensi format keluaran serta validitas data yang digunakan dalam proses evaluasi dan visualisasi.

2.7 Metode Evaluasi

Tahap pengujian dan evaluasi dilakukan untuk menilai kinerja sistem dalam menghasilkan estimasi beban tugas berbasis *Retrieval-Augmented Generation* (RAG). Evaluasi dilakukan terhadap tiga aspek utama, yaitu *retrieval*, hasil estimasi, dan *reasoning*. Setiap aspek diuji menggunakan pendekatan yang berbeda sesuai dengan karakteristik keluaran yang dihasilkan oleh sistem.

2.7.1 Evaluasi Hasil Retrieval (Hit Rate dan Precision Rate)

Evaluasi hasil retrieval dilakukan untuk mengukur kemampuan sistem dalam menemukan data historis yang relevan terhadap query yang diberikan pengguna. Pengujian dilakukan menggunakan 10 query tugas akademik baru yang tidak terdapat pada dataset historis. Pendekatan ini dipilih agar evaluasi dapat merepresentasikan kondisi penggunaan sistem secara nyata.

1. Hit Rate

Hit Rate digunakan untuk mengukur kemampuan sistem dalam menemukan setidaknya satu data yang relevan dalam hasil *retrieval*. Metrik ini berfokus pada

keberhasilan sistem dalam menyediakan konteks yang dapat digunakan oleh model, tanpa mempertimbangkan jumlah total hasil relevan yang ditemukan.

Penggunaan *Hit Rate* dalam penelitian ini didasarkan pada karakteristik RAG, di mana keberadaan minimal satu konteks yang relevan sudah dapat membantu model dalam menghasilkan jawaban yang lebih akurat. Oleh karena itu, *Hit Rate* menjadi indikator awal untuk menilai apakah sistem mampu menemukan informasi yang sesuai dengan kebutuhan query pengguna.

Dalam penelitian ini, evaluasi retrieval dilakukan dengan menggunakan tiga hasil retrieval teratas. Oleh karena itu, metrik yang digunakan adalah *Hit Rate@3* (HR@3). *Hit Rate@3* mengukur proporsi query yang berhasil memperoleh minimal satu hasil relevan pada tiga hasil retrieval teratas yang diberikan sistem.

Secara matematis, *Hit Rate@3* dirumuskan pada persamaan (4) sebagai berikut:

$$H R@3 = \frac{\text{Jumlah query dengan minimal satu hasil relevan pada tugas retrieval}}{\text{Total jumlah query}} \quad (4)$$

Namun demikian, *Hit Rate* tidak mempertimbangkan jumlah total data relevan yang ditemukan, sehingga tidak dapat digunakan untuk menilai kualitas keseluruhan hasil *retrieval*.

2. Precision Rate

Precision digunakan untuk mengukur kualitas hasil *retrieval*, yaitu proporsi data yang relevan dibandingkan dengan seluruh data yang diambil oleh sistem. Dalam konteks *retrieval*, metrik ini dinyatakan sebagai *Precision@k*, yang berfokus pada kualitas hasil pada sejumlah data teratas yang digunakan sebagai konteks.

Penggunaan *Precision* dalam penelitian ini bertujuan untuk memastikan bahwa hasil yang diambil oleh sistem tidak hanya mengandung data relevan, tetapi juga meminimalkan keberadaan data yang tidak relevan. Hal ini penting dalam RAG, karena konteks yang tidak relevan dapat menurunkan kualitas *reasoning* dan keluaran yang dihasilkan oleh model.

Karena dalam penelitian ini jumlah tugas yang diambil adalah tiga, maka nilai *precision* dapat dirumuskan menjadi *Precision@3* dapat dirumuskan sebagai berikut:

$$Precision@3 = \frac{\text{Jumlah hasil relevan pada 3 tugas retrieval}}{\text{Total tugas yang diambil}} \quad (5)$$

Evaluasi *retrieval* dalam penelitian ini dilakukan melalui tahapan sebagai berikut :

1. Menyiapkan 10 *query* yang akan diuji untuk mensimulasikan variasi input pengguna.

2. Memasukkan setiap *query* ke dalam sistem untuk menghasilkan hasil *retrieval* sebanyak 3 tugas dengan kemiripan tertinggi.
3. Melakukan penilaian terhadap setiap hasil *retrieval* apakah termasuk relevan atau tidak, dengan ketentuan.
 - Relevan: Dalam penelitian ini, hasil *retrieval* dinyatakan relevan apabila memiliki kesamaan aktivitas yang dilakukan mahasiswa dengan tugas baru. Kesamaan aktivitas dipilih sebagai kriteria utama karena secara langsung merepresentasikan jenis usaha dan proses pengerjaan yang diperlukan, sehingga hasil *retrieval* yang memenuhi kriteria ini dapat digunakan sebagai referensi pembandingan dalam mengestimasi *deadline*, waktu pengerjaan, dan tingkat kesulitan tugas.
 - Tidak Relevan: Hasil *retrieval* dianggap tidak relevan apabila tidak memiliki kesamaan aktivitas utama yang dilakukan mahasiswa dengan tugas baru. Akibatnya, hasil *retrieval* tersebut tidak dapat digunakan secara memadai sebagai referensi pembandingan dalam mengestimasi *deadline*, waktu pengerjaan, maupun tingkat kesulitan tugas, meskipun masih berada dalam ranah akademik yang sama.
4. Menentukan nilai *Hit Rate* dengan menganggap suatu *query* sebagai *hit* apabila terdapat minimal satu hasil relevan dalam tugas *retrieval*.
5. Menghitung nilai *Precision@3* dengan membandingkan jumlah hasil relevan dengan total hasil dalam tugas hasil *retrieval*.
6. Mengulangi proses evaluasi untuk seluruh *query* yang digunakan dalam pengujian.
7. Menghitung rata-rata nilai *Hit Rate* dan *Precision* dari seluruh *query* untuk memperoleh nilai evaluasi keseluruhan sistem *retrieval*.

2.7.2 Evaluasi Kelayakan Hasil Estimasi

Evaluasi manual dilakukan untuk menilai kewajaran hasil estimasi yang dihasilkan sistem berdasarkan perspektif mahasiswa. Evaluasi dilakukan menggunakan Google Form dengan melibatkan 10 mahasiswa sebagai responden dan menggunakan 10 tugas akademik yang telah diestimasi oleh sistem. Untuk mengurangi beban penilaian, responden dibagi ke dalam dua paket evaluasi. Paket A berisi tugas-tugas umum Sistem Informasi dan tugas yang berfokus pada bidang Manajemen Sistem Informasi, sedangkan Paket B berisi tugas-tugas umum Sistem Informasi dan tugas yang berfokus pada bidang Sistem Informasi Cerdas. Setiap responden hanya diminta mengevaluasi 5 hasil estimasi tugas sesuai paket yang diterima. Pada setiap tugas, responden diberikan deskripsi tugas beserta hasil estimasi sistem yang mencakup rentang *deadline*, rentang lama pengerjaan, dan tingkat kesulitan (*difficulty*). Selanjutnya, responden diminta memberikan penilaian menggunakan skala Likert 1–5 terhadap kewajaran hasil estimasi yang diberikan sistem.

Aspek yang dievaluasi meliputi:

1. Kesesuaian estimasi *deadline* dengan kompleksitas tugas.
2. Kerealistisan estimasi lama pengerjaan tugas.

3. Kesesuaian tingkat kesulitan (*difficulty*) dengan beban tugas yang diberikan. Hasil evaluasi manual ini digunakan untuk mengetahui apakah estimasi yang dihasilkan sistem sudah dianggap realistis dan sesuai dengan persepsi mahasiswa terhadap beban tugas akademik.

2.7.3 Evaluasi Kualitas Reasoning Menggunakan LLM-as-a-Judge

Sebelumnya telah dilakukan evaluasi menggunakan manual judgement mahasiswa terhadap tiga hasil estimasi berbentuk kuantitatif, yaitu rentang *deadline*, rentang lama pengerjaan, dan *difficulty*. Selanjutnya, penelitian ini juga melakukan evaluasi terhadap kualitas reasoning yang dihasilkan sistem menggunakan pendekatan *LLM-as-a-Judge*. Evaluasi ini bertujuan untuk menilai apakah alasan estimasi yang dihasilkan sistem memanfaatkan tugas retrieval, memiliki korelasi dengan hasil estimasi, dan mudah dipahami.

Pada proses evaluasi, model GPT-5.5 digunakan sebagai evaluator (*judge model*) karena memiliki kemampuan penalaran dan evaluasi teks yang baik. Model menerima empat komponen masukan, yaitu deskripsi tugas baru, referensi historis hasil *retrieval*, hasil estimasi sistem, dan reasoning yang dihasilkan sistem. Berdasarkan informasi tersebut, model melakukan penilaian terhadap kualitas reasoning secara otomatis menggunakan prompt evaluasi yang telah dirancang khusus.

Input evaluasi terdiri atas:

- Deskripsi tugas baru yang dianalisis.
- Referensi historis hasil retrieval yang digunakan sistem.
- Hasil estimasi sistem yang mencakup rentang *deadline*, rentang lama pengerjaan, dan tingkat kesulitan.
- *Reasoning* yang dihasilkan sistem sebagai dasar pemberian estimasi.

Evaluator kemudian diminta menilai kualitas *reasoning* berdasarkan tiga aspek utama sebagai berikut.:

1. Pemanfaatan Referensi Historis (*Historical Reference Utilization*)

Aspek ini menilai sejauh mana *reasoning* memanfaatkan referensi historis yang diperoleh melalui proses retrieval sebagai dasar analisis terhadap tugas baru. Penilaian difokuskan pada apakah referensi historis digunakan sebagai pembandingan yang relevan serta mampu mendukung proses penalaran yang dilakukan sistem. Selain itu, aspek ini juga mempertimbangkan apakah reasoning menunjukkan hubungan yang jelas antara tugas baru dan referensi historis tanpa mengharuskan pembahasan seluruh referensi secara rinci.

2. Keterkaitan Reasoning dengan Hasil Estimasi (*Estimation Consistency*)

Aspek ini menilai kesesuaian antara *reasoning* yang dihasilkan dan hasil estimasi yang diberikan sistem. Penilaian dilakukan dengan melihat apakah reasoning mampu memberikan alasan yang mendukung estimasi *deadline*, lama pengerjaan, dan tingkat kesulitan yang dihasilkan. *Reasoning* dianggap baik apabila analisis yang disampaikan selaras dengan hasil estimasi serta

mampu menjelaskan hubungan antara karakteristik tugas, referensi historis yang digunakan, dan estimasi yang dihasilkan.

3. Kejelasan dan Koherensi Reasoning (*Reasoning Clarity and Coherence*)

Aspek ini menilai kualitas penyampaian *reasoning* dari sisi keterbacaan dan struktur penjelasan. Penilaian difokuskan pada apakah *reasoning* disampaikan secara jelas, runtut, mudah dipahami, dan tidak mengandung pengulangan informasi yang tidak diperlukan. *Reasoning* yang baik mampu menjelaskan dasar pertimbangan sistem secara ringkas namun tetap informatif sehingga memudahkan pengguna memahami hasil estimasi yang diberikan.

Selain menghasilkan skor untuk masing-masing aspek, evaluator juga menghasilkan *overall score* yang merupakan rata-rata dari seluruh aspek penilaian serta *overall judgement* berupa ringkasan kualitas *reasoning* secara keseluruhan. Struktur keluaran evaluasi ditampilkan dalam format JSON agar dapat diproses secara otomatis oleh sistem.

Penilaian pada setiap aspek dilakukan menggunakan skala 1–5, di mana 1 menunjukkan kualitas reasoning yang sangat buruk dan 5 menunjukkan kualitas reasoning yang sangat baik. Melalui evaluasi ini, penelitian dapat menilai sejauh mana *reasoning* yang dihasilkan sistem tidak hanya konsisten dengan hasil estimasi yang diberikan, tetapi juga mampu memanfaatkan referensi historis secara tepat dan menghasilkan penjelasan yang logis dalam konteks estimasi beban tugas akademik berbasis *Retrieval-Augmented Generation* (RAG).

2.7.4 Evaluasi Parsing Success Rate (PSR)

Setiap tugas baru yang dimasukkan ke dalam sistem akan menghasilkan keluaran berupa JSON yang selanjutnya akan melakukan parsing sebelum ditampilkan pada dashboard, untuk itu perlu juga melakukan evaluasi terhadap keberhasilan sistem dalam menghasilkan keluaran yang dapat ditampilkan pada dashboard menggunakan *Parsing Success Rate* (PSR). Evaluasi ini bertujuan untuk mengukur tingkat konsistensi sistem dalam menghasilkan struktur keluaran JSON yang valid sehingga hasil estimasi dapat diproses dan ditampilkan oleh aplikasi tanpa mengalami kegagalan parsing.

Evaluasi PSR dilakukan menggunakan 50 query tugas baru. Setiap query diproses oleh sistem untuk mengukur tingkat keberhasilan model dalam menghasilkan keluaran yang memenuhi kriteria PSR. Suatu keluaran dinyatakan berhasil apabila memenuhi seluruh kondisi berikut.

1. Menghasilkan struktur JSON yang valid dan dapat diparsing oleh sistem.
2. Seluruh informasi estimasi berhasil ditampilkan pada dashboard, yang meliputi rentang deadline, estimasi waktu pengerjaan, tingkat kesulitan, penjelasan estimasi (*reasoning*), dan referensi tugas hasil retrieval.
3. Memenuhi seluruh aturan yang telah ditetapkan pada prompt, antara lain:
 - deadline mempertimbangkan estimasi waktu pengerjaan berdasarkan asumsi mahasiswa mengerjakan tugas sekitar 1–2 jam per hari sebagai pedoman kewajaran;

- deadline minimum tidak lebih pendek dari waktu yang dibutuhkan berdasarkan asumsi tersebut;
- selisih antara *deadline_min_days* dan *deadline_max_days* tidak melebihi 7 hari;
- selisih antara *est_time_min_hours* dan *est_time_max_hours* tidak melebihi 40% dari *est_time_min_hours*.

Sebaliknya, keluaran dinyatakan gagal apabila salah satu kondisi tersebut tidak terpenuhi, baik karena keluaran tidak dapat diparsing menjadi JSON yang valid (menampilkan pesan kesalahan berupa “Gagal parsing JSON dari model”) maupun karena melanggar aturan yang telah ditetapkan pada prompt.

Nilai Parsing Success Rate sistem dapat dihitung menggunakan persamaan (7) berikut.

$$PSR = \frac{\text{jumlah keluaran yang memenuhi seluruh kriteria PSR}}{\text{total query uji}} \quad (6)$$

BAB III HASIL DAN PEMBAHASAN

3.1 Implementasi Sistem

Pada penelitian ini, sistem estimasi beban tugas mahasiswa berbasis *Retrieval-Augmented Generation* (RAG) berhasil diimplementasikan sesuai dengan perancangan yang telah dijelaskan pada Bab II. Sistem yang dikembangkan mampu mengintegrasikan proses *semantic retrieval* dengan model bahasa generatif untuk menghasilkan estimasi beban tugas secara otomatis dan terstruktur.

Implementasi sistem dilakukan dengan memanfaatkan dataset tugas akademik yang telah melalui proses validasi dan pembersihan data. Dataset akhir yang digunakan berjumlah 278 data, yang terdiri dari deskripsi tugas akademik (*task_teks*) sebagai data utama serta tiga atribut target, yaitu estimasi tenggat waktu (*deadline_days*), estimasi lama pengerjaan (*est_time_hours*), dan tingkat kesulitan (*difficulty*). Data tersebut menjadi dasar dalam membangun representasi semantik dan referensi historis yang digunakan dalam proses estimasi.

Sistem yang dikembangkan bekerja melalui dua tahapan utama, yaitu proses offline dan proses online. Pada tahap offline, seluruh data historis dikonversi menjadi representasi vektor menggunakan model *SentenceTransformer multilingual-e5-base*. Hasil embedding tersebut kemudian disimpan dalam bentuk indeks vektor menggunakan FAISS, sehingga memungkinkan proses pencarian kemiripan dilakukan secara efisien. Selain itu, metadata terkait tugas juga disimpan untuk menjaga keterkaitan antara vektor dan informasi aslinya.

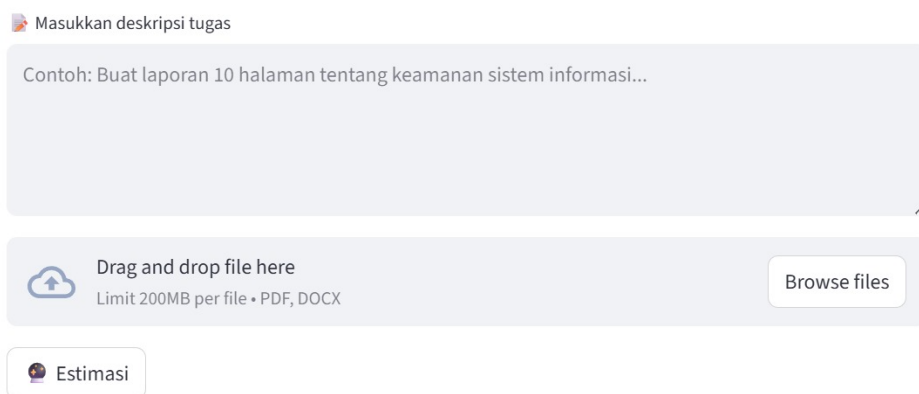
Selanjutnya, pada tahap online, sistem menerima input berupa deskripsi tugas baru dari pengguna melalui dashboard. Input tersebut kemudian diproses melalui beberapa tahapan, yaitu konversi teks menjadi embedding, pencarian kemiripan terhadap data historis dalam indeks FAISS, serta pengambilan sejumlah data dengan tingkat kemiripan tertinggi (*retrieval*). Dalam penelitian ini, jumlah tugas dengan kemiripan tertinggi ditetapkan sebesar tiga, sehingga sistem menggunakan tiga referensi tugas historis sebagai konteks tambahan dalam proses estimasi.

Hasil retrieval tersebut kemudian digunakan untuk membangun prompt yang diberikan kepada model bahasa generatif. Model yang digunakan adalah LLaMa 3.1 8B yang diakses melalui API Groq. Model ini berperan dalam menganalisis kompleksitas tugas berdasarkan deskripsi input dan konteks historis, serta menghasilkan estimasi beban tugas dalam format JSON terstruktur. Keluaran yang dihasilkan mencakup nilai estimasi rentang tenggat waktu, rentang lama pengerjaan, tingkat kesulitan, serta penjelasan (*reasoning*) yang mendukung hasil estimasi.

Untuk memastikan keluaran sistem dapat diproses secara konsisten, dilakukan mekanisme parsing terhadap keluaran model. Proses ini bertujuan untuk mengekstrak struktur JSON yang valid serta menghindari kesalahan format yang dapat mengganggu proses evaluasi maupun visualisasi.

Seluruh proses dalam sistem diintegrasikan ke dalam sebuah dashboard berbasis Streamlit yang berfungsi sebagai antarmuka pengguna. Melalui dashboard ini, pengguna dapat memasukkan deskripsi tugas akademik yang akan dianalisis oleh sistem. Setelah proses estimasi selesai, dashboard akan menampilkan hasil berupa estimasi tenggat waktu, estimasi lama pengerjaan, tingkat kesulitan tugas, serta penjelasan yang mendasari hasil estimasi tersebut. Selain itu, informasi referensi historis yang digunakan dalam proses retrieval juga dapat ditampilkan untuk memberikan transparansi terhadap proses pengambilan keputusan sistem. Tampilan dashboard yang dihasilkan pada penelitian ini ditunjukkan pada **Gambar 7**.

Sistem Estimasi Beban Tugas



Masukkan deskripsi tugas

Contoh: Buat laporan 10 halaman tentang keamanan sistem informasi...

Drag and drop file here
Limit 200MB per file • PDF, DOCX

Browse files

Estimasi

Gambar 7. Tampilan awal sistem

Seperti yang ditunjukkan pada **Gambar 7**, ketika pengguna pertama kali mengakses dashboard, sistem menampilkan teks area input deskripsi tugas dalam bentuk teks dan tombol upload dokumen untuk inputan berupa dokumen (DOCX/PDF), serta tombol untuk memulai proses estimasi. Berikut rincian mengenai fitur yang dimiliki oleh sistem :

a. Teks Area Input Deskripsi Tugas

Pada bagian awal dashboard terdapat komponen teks area yang digunakan untuk memasukkan deskripsi tugas akademik langsung berupa teks. Pengguna dapat menuliskan deskripsi secara bebas, baik dalam bentuk poin-poin maupun narasi lengkap yang mencakup ruang lingkup pekerjaan, kebutuhan teknis, serta keluaran yang diharapkan. Selain itu, area teks digunakan sebagai tempat pratinjau dan penyuntingan teks hasil ekstraksi dokumen, dengan batas maksimum 3.000 karakter.

b. Upload Dokumen

Fitur upload dokumen memungkinkan pengguna untuk menggunakan dokumen tugas berupa DOCX/PDF secara langsung tanpa perlu menyalin isi dari file terkait sehingga memberikan kemudahan bagi pengguna. Upload Dokumen

menyajikan dua metode upload seperti pada umumnya yakni dengan menggunakan tombol dan menggeser dokumen ke tempat yang telah tersedia.

Hasil upload dokumen akan ditampilkan kembali dalam bentuk teks pada fitur teks area untuk dilakukan pratinjau ulang kepada pengguna apabila ingin melakukan pengeditan terhadap tugas dari dokumen terupload.

c. Tombol Estimasi

Tombol estimasi berfungsi sebagai tombol eksekusi yang apabila ditekan akan melakukan proses estimasi terhadap tugas yang telah dimasukkan. Tombol estimasi dilengkapi dengan pengecekan inputan, apabila tidak ada inputan yang dimasukkan dan pengguna tetap menekan tombol estimasi, maka sebuah peringatan pengisian inputan akan ditampilkan pada dashboard sebagai timbal balik dari sistem.

Setelah inputan dieksekusi oleh sistem, sistem akan menampilkan keluaran tampilan berupa hasil estimasi berupa hasil estimasi, penjelasan estimasi, dan referensi tugas yang diambil. Untuk lebih lengkapnya, tampilan hasil estimasi ditunjukkan pada **Gambar 7** dan **Gambar 8** di bawah

Sistem Estimasi Beban Tugas

 Masukkan deskripsi tugas

TUGAS INDIVIDU — Pemrograman Web

Buat halaman web profil diri sendiri menggunakan HTML dan Bootstrap 5 — tidak perlu CSS tambahan yang berlebihan, manfaatkan komponen Bootstrap yang sudah ada.

Yang harus ada:



Drag and drop file here

Limit 200MB per file • PDF, DOCX

Browse files


Estimasi

Deadline (hari)	Difficulty	Estimasi Jam
5 - 12	2	10 - 14

>
📌 Penjelasan Estimasi

>
📖 Referensi Tugas Serupa

Gambar 8. Tampilan hasil estimasi sistem

Dalam dashboard, hasil estimasi diletakkan di bawah tombol estimasi sebagai tampilan tambahan sehingga memungkinkan penginputan tugas ulang. Berikut

merupakan detail fitur tampilan tambahan yang disajikan oleh sistem sebagai hasil estimasi dan informasi tambahan untuk memperkuat estimasi :

a. Keluaran Hasil Estimasi

Keluaran hasil estimasi menyajikan bagian utama dalam sistem, yakni keluaran estimasi dari sistem. Keluaran hasil estimasi disajikan dalam tiga variabel yaitu deadline (hari), difficulty (skala 1-5), dan estimasi jam pengerjaan tugas sesuai dengan yang telah dijelaskan dalam metode penelitian. Ketiga variabel ini disusun dengan mendatar untuk memudahkan pengguna dalam membaca dan membandingkan ketiga variabel tersebut karena dapat dilihat sekaligus.

b. Penjelasan Estimasi

Penjelasan estimasi yang muncul di bawah keluaran hasil estimasi juga merupakan hasil *generate* dari ILM yang dilempar oleh sistem. Penjelasan ini memuat penjelasan singkat mengenai alasan pemberian nilai estimasi pada keluaran hasil estimasi. Fitur penjelasan estimasi disajikan dalam bentuk *collapsible panel* (accordion) yang dapat diperluas dan disembunyikan sesuai kebutuhan pengguna. Penyajian penjelasan estimasi bertujuan untuk meningkatkan transparansi serta membantu pengguna memahami dasar pertimbangan sistem dalam menghasilkan estimasi.

c. Referensi Tugas Serupa

Referensi tugas serupa merupakan bagian yang menampilkan tugas-tugas yang menjadi hasil pada bagian *retrieval* dan tugas yang dioperkan kepada LLM. Fitur ini bertujuan untuk meningkatkan transparansi dan validitas hasil estimasi melalui penyajian tugas-tugas dengan tingkat kemiripan yang relevan. Referensi tugas serupa diletakkan di bawah penjelasan estimasi pada dashboard. Fitur ini juga disajikan dalam bentuk *collapsible panel* (accordion) yang di dalamnya disusun dalam format tabel yang memudahkan pengguna untuk melihat referensi yang dijadikan dasar estimasi.

Sistem Estimasi Beban Tugas

Masukkan deskripsi tugas

TUGAS INDIVIDU — Pemrograman Web

Buat halaman web profil diri sendiri menggunakan HTML dan Bootstrap 5 — tidak perlu CSS tambahan yang berlebihan, manfaatkan komponen Bootstrap yang sudah ada.

Yang harus ada:



Drag and drop file here

Limit 200MB per file • PDF, DOCX

Browse files



Estimasi

Deadline (hari)

Difficulty

Estimasi Jam

5 - 12

2

10 - 14

Penjelasan Estimasi

Tugas baru ini meminta mahasiswa untuk membuat halaman web profil diri sendiri menggunakan HTML dan Bootstrap 5. Referensi historis yang relevan adalah 'Halaman Profil' karena keduanya memiliki tujuan yang sama, yaitu membuat halaman web profil diri. Namun, tugas baru ini memiliki beberapa perbedaan, seperti penggunaan Bootstrap 5 dan adanya kebutuhan untuk membuat navbar yang berfungsi di mobile. Oleh karena itu, tugas baru ini dapat dianggap lebih kompleks daripada 'Halaman Profil'. Berdasarkan estimasi waktu pengerjaan, deadline minimum yang realistis adalah 5 hari, sedangkan deadline maximum adalah 12 hari. Estimasi waktu pengerjaan bagi mahasiswa yang memahami materi dengan baik adalah 10 jam, sedangkan bagi mahasiswa dengan kemampuan rata-rata adalah 14 jam. Tingkat kesulitan tugas ini dapat dianggap 2 karena memiliki beberapa kebutuhan yang kompleks, seperti membuat navbar yang berfungsi di mobile dan menggunakan Bootstrap 5.

Referensi Tugas Serupa

		Similarity Score	Deadline (hari)	Difficulty	Estimasi (jam)
0	an HTML dan CSS. Ketentuan: Sections: H	0.8751	7	2	
1	Web Deskripsi Tugas: Buat halaman HTML	0.8681	5	1	
2	an HTML dan CSS murni (tanpa framework)	0.8534	7	3	

Gambar 9. Tampilan hasil estimasi sistem (2)

3.2 Evaluasi Sistem

Evaluasi sistem dilakukan untuk menjawab rumusan masalah kedua, yaitu bagaimana kinerja sistem *Retrieval-Augmented Generation* (RAG) yang dikembangkan dalam menghasilkan estimasi beban tugas berdasarkan hasil keluaran sistem. Evaluasi dilakukan dengan menguji setiap komponen sistem, meliputi proses retrieval, hasil estimasi, reasoning, dan parsing keluaran. Evaluasi pada komponen retrieval dilakukan menggunakan metrik *Hit Rate* dan *Precision Rate* untuk mengukur kemampuan sistem dalam mengambil referensi yang relevan. Evaluasi pada hasil estimasi dilakukan dengan penilaian manual (*manual judgement*) oleh mahasiswa. Evaluasi reasoning dilakukan menggunakan pendekatan LLM-as-a-Judge, sedangkan evaluasi parsing keluaran dilakukan menggunakan metrik *Parsing Success Rate* (PSR).

3.2.1 Evaluasi Retrieval

Evaluasi retrieval dilakukan untuk mengukur kemampuan sistem dalam menemukan data historis yang relevan terhadap query tugas yang diberikan pengguna. Pengujian dilakukan menggunakan 10 query tugas akademik baru yang tidak terdapat pada dataset historis. Penggunaan query baru bertujuan untuk merepresentasikan kondisi penggunaan sistem secara nyata, yaitu ketika pengguna memasukkan deskripsi tugas yang belum pernah ada sebelumnya. Setiap query menghasilkan tiga hasil retrieval teratas yang kemudian dievaluasi berdasarkan tingkat relevansinya.

Penilaian relevansi dilakukan dengan mengelompokkan setiap hasil retrieval ke dalam dua kategori, yaitu relevan dan tidak relevan. Suatu hasil retrieval dinyatakan relevan apabila masih dapat dijadikan referensi pembanding dalam proses estimasi beban tugas berdasarkan kesamaan aktivitas utama yang dilakukan mahasiswa. Sebaliknya, hasil retrieval dinyatakan tidak relevan apabila tidak memiliki kesamaan aktivitas yang cukup sehingga kurang mendukung proses estimasi. Hasil evaluasi retrieval ditunjukkan pada **Tabel 9**.

Tabel 9. Hasil Evaluasi Retrieval

Query	Peringkat 1	Peringkat 2	Peringkat 3	Jumlah Relevan	Hit/Not
1	Relevan	Relevan	Tidak Relevan	2	Hit
2	Relevan	Relevan	Tidak Relevan	2	Hit
3	Relevan	Relevan	Tidak Relevan	2	Hit
4	Relevan	Relevan	Relevan	3	Hit
5	Relevan	Relevan	Tidak Relevan	2	Hit
6	Relevan	Relevan	Relevan	3	Hit
7	Relevan	Relevan	Relevan	3	Hit
8	Relevan	Relevan	Relevan	3	Hit
9	Relevan	Relevan	Relevan	3	Hit

Query	Peringkat 1	Peringkat 2	Peringkat 3	Jumlah Relevan	Hit/Not
10	Relevan	Tidak Relevan	Tidak Relevan	1	Hit

Berdasarkan hasil evaluasi retrieval pada **Tabel 9**, seluruh query berhasil memperoleh minimal satu hasil retrieval yang relevan pada 3 tugas dengan kemiripan tertinggi. Oleh karena itu, nilai *Hit Rate* yang diperoleh sistem adalah:

$$HR@3 = \frac{\text{Jumlah query dengan minimal satu hasil relevan}}{\text{Total jumlah query}} = \frac{10}{10} = 1.0 = 100 \%$$

Hasil tersebut menunjukkan bahwa sistem retrieval mampu menemukan setidaknya satu konteks yang relevan untuk seluruh query yang diuji. Dalam konteks *Retrieval-Augmented Generation* (RAG), hasil ini menunjukkan bahwa sistem telah mampu menyediakan konteks yang dapat membantu model dalam menghasilkan estimasi dan reasoning yang sesuai dengan tugas pengguna.

Selain *Hit Rate*, evaluasi juga dilakukan menggunakan *Precision@3* untuk mengukur kualitas hasil retrieval yang diperoleh sistem. Dari total 30 data retrieval yang dihasilkan pada seluruh query, sebanyak 24 data dinilai relevan karena memiliki kesamaan aktivitas. Perhitungan *Precision@3* dilakukan sebagai berikut:

$$\begin{aligned} Precision@3 &= \frac{\text{jumlah data relevan diambil}}{\text{Total data yang diambil}} \\ &= \frac{24}{30} = 0.8 = 80\% \end{aligned}$$

Berdasarkan hasil evaluasi retrieval, sistem memperoleh nilai *Hit Rate@3* sebesar 100% dan *Precision@3* sebesar 80%. Hasil ini menunjukkan bahwa seluruh query yang diuji berhasil memperoleh setidaknya satu referensi historis yang relevan, sementara sebagian besar hasil retrieval yang diberikan sistem juga memiliki keterkaitan dengan tugas yang dicari.

Hasil tersebut menunjukkan bahwa komponen retrieval pada pendekatan *Retrieval-Augmented Generation* (RAG) mampu menemukan konteks historis yang relevan dari variasi deskripsi tugas yang diuji. Referensi historis yang diperoleh memberikan informasi pembandingan bagi model bahasa dalam memahami karakteristik tugas, sehingga proses estimasi tidak hanya bergantung pada pengetahuan umum model, tetapi juga mempertimbangkan pola dan karakteristik tugas akademik yang terdapat dalam dataset penelitian.

3.2.2 Evaluasi Manual Judgement

Evaluasi *manual judgement* dilakukan untuk menilai kelayakan hasil estimasi yang dihasilkan sistem berdasarkan perspektif mahasiswa. Pengujian melibatkan 10 mahasiswa sistem informasi sebagai responden dengan menggunakan 10 query

tugas akademik yang berbeda. Evaluasi dibagi menjadi dua paket sehingga setiap mahasiswa hanya memberikan penilaian untuk 5 query tugas. Mahasiswa akan menilai hasil estimasi sistem yang meliputi rentang *deadline*, rentang lama pengerjaan, dan *difficulty* menggunakan skala Likert 1–5.

Aspek yang dievaluasi meliputi kesesuaian estimasi *deadline* terhadap kompleksitas tugas, kerealistisan estimasi waktu pengerjaan, serta kesesuaian tingkat *difficulty* terhadap beban tugas yang diberikan. Hasil evaluasi *manual judgement* untuk setiap tugas ditunjukkan pada **Tabel 10**, sedangkan rata-rata penilaian pada setiap aspek evaluasi ditunjukkan pada **Tabel 11**.

Tabel 10. Hasil Evaluasi Manual Judgement Setiap Tugas

No	Query Tugas	Kesesuaian Deadline	Realisme Lama Pengerjaan	Kesesuaian Difficulty
1	Tugas 1	4,4	4,4	4,2
2	Tugas 2	4,2	4,2	4,6
3	Tugas 3	4,4	4,0	4,4
4	Tugas 4	4,4	4,2	4,0
5	Tugas 5	4,6	4,4	4,0
6	Tugas 6	4,2	3,6	4,2
7	Tugas 7	4,2	3,8	2,6
8	Tugas 8	3,4	4,2	3,6
9	Tugas 9	4,2	3,6	4,6
10	Tugas 10	3,6	3,0	3,6

Tabel 11. Rata-Rata Setiap Aspek Evaluasi

Aspek Evaluasi	Rata-rata
Kesesuaian Deadline	4,16
Realisme Lama Pengerjaan	3,94
Kesesuaian Difficulty	3,98

Berdasarkan hasil evaluasi pada **Tabel 11**, aspek kesesuaian *deadline* memperoleh rata-rata tertinggi sebesar 4,16. Hasil ini menunjukkan bahwa estimasi tenggat waktu yang dihasilkan sistem secara umum dinilai telah sesuai dengan kompleksitas tugas akademik. Aspek kesesuaian *difficulty* memperoleh rata-rata sebesar 3,98, yang mengindikasikan bahwa tingkat kesulitan yang dihasilkan sistem telah cukup sesuai dengan persepsi mahasiswa terhadap beban tugas yang diberikan. Sementara itu, aspek realisme lama pengerjaan memperoleh rata-rata sebesar 3,94. Nilai tersebut menunjukkan bahwa estimasi waktu pengerjaan yang dihasilkan sistem masih tergolong realistis, meskipun pada beberapa tugas terdapat perbedaan persepsi antarresponden mengenai waktu yang dibutuhkan untuk menyelesaikan tugas.

Secara keseluruhan, hasil evaluasi *manual judgement* menunjukkan bahwa sistem mampu menghasilkan estimasi yang layak dan realistis berdasarkan penilaian mahasiswa. Seluruh aspek evaluasi memperoleh nilai rata-rata mendekati atau melebihi 4 dari skala 5, yang mengindikasikan tingkat penerimaan yang baik terhadap hasil estimasi yang diberikan sistem.

3.2.3 Evaluasi LLM-as-a-Judge

Evaluasi *LLM-as-a-Judge* dilakukan untuk menilai kualitas *reasoning* yang dihasilkan sistem estimasi beban tugas akademik berbasis *Retrieval-Augmented Generation* (RAG). Evaluasi ini bertujuan untuk mengetahui apakah *reasoning* yang dihasilkan sistem mampu memanfaatkan referensi historis sebagai pembandingan, memiliki kaitan dengan hasil estimasi yang diberikan, serta jelas dan mudah dipahami.

Pengujian dilakukan terhadap 10 sampel hasil estimasi yang dihasilkan sistem. Pada evaluasi ini, model GPT-5.5 digunakan sebagai evaluator (*LLM-as-a-Judge*) untuk memberikan penilaian terhadap tiga aspek utama, yaitu pemanfaatan referensi historis, konsistensi *reasoning* dengan hasil estimasi, serta kewajaran dan kelogisan *reasoning*. Penilaian dilakukan menggunakan skala 1–5, di mana nilai yang lebih tinggi menunjukkan kualitas *reasoning* yang lebih baik.

Tabel 12. Hasil Evaluasi LLM-as-a-Judge

Aspek Evaluasi	Rata-rata Skor
Pemanfaatan referensi historis	3,60
Keterkaitan reasoning dengan hasil estimasi	4,10
Kejelasan dan koherensi reasoning	4,50
Rata-rata keseluruhan	4,07

Berdasarkan hasil evaluasi pada **Tabel 12**, aspek Kejelasan dan Koherensi Reasoning memperoleh nilai rata-rata tertinggi sebesar 4,50. Hasil ini menunjukkan bahwa reasoning yang dihasilkan sistem umumnya telah disusun secara jelas, runtut, dan mudah dipahami. Penjelasan yang diberikan mampu menggambarkan dasar pertimbangan estimasi tanpa menimbulkan ambiguitas yang berarti, sehingga memudahkan pengguna dalam memahami alasan di balik hasil estimasi yang dihasilkan sistem.

Aspek Keterkaitan Reasoning dengan Hasil Estimasi memperoleh nilai rata-rata sebesar 4,10. Nilai tersebut menunjukkan bahwa reasoning yang dihasilkan sebagian besar telah mendukung hasil estimasi yang diberikan sistem. Karakteristik tugas yang dijelaskan dalam reasoning umumnya selaras dengan nilai estimasi *deadline*, lama pengerjaan, maupun tingkat kesulitan yang dihasilkan. Meskipun demikian, masih terdapat beberapa query yang memperoleh nilai lebih rendah karena ditemukan ketidaksesuaian antara penjelasan yang diberikan dengan hasil estimasi yang dihasilkan sistem.

Sementara itu, aspek Pemanfaatan Referensi Historis memperoleh nilai rata-rata terendah yaitu 3,60. Hasil ini menunjukkan bahwa meskipun sistem telah memanfaatkan referensi historis dalam proses *reasoning*, tingkat pemanfaatannya masih bervariasi pada beberapa query. Pada sejumlah kasus, referensi historis telah digunakan secara eksplisit sebagai pembanding terhadap tugas baru, sedangkan pada kasus lainnya referensi hanya digunakan secara umum atau bahkan tidak dijelaskan secara jelas dalam reasoning yang dihasilkan.

Secara keseluruhan, evaluasi *LLM-as-a-Judge* menghasilkan rata-rata nilai 4,07. Hasil ini menunjukkan bahwa reasoning yang dihasilkan sistem telah mampu menjelaskan dasar estimasi secara cukup konsisten, jelas, dan mudah dipahami. Selain itu, penggunaan pendekatan *Retrieval-Augmented Generation* (RAG) memungkinkan sistem tidak hanya menghasilkan nilai estimasi berupa deadline, lama pengerjaan, dan tingkat kesulitan, tetapi juga menyediakan reasoning yang menjelaskan pertimbangan di balik estimasi tersebut. Dengan demikian, hasil estimasi yang diberikan menjadi lebih transparan dan dapat dipahami oleh pengguna dibandingkan jika sistem hanya menghasilkan nilai estimasi tanpa penjelasan pendukung.

3.2.4 Evaluasi Parsing Success Rate

Evaluasi *Parsing Success Rate* (PSR) dilakukan untuk mengukur kemampuan sistem dalam menghasilkan keluaran yang tidak hanya sesuai dengan format JSON yang telah ditentukan, tetapi juga memenuhi seluruh aturan dan batasan yang ditetapkan pada prompt. Pengujian ini penting karena hasil estimasi yang dihasilkan model bahasa akan diproses secara otomatis oleh sistem sebelum ditampilkan pada dashboard aplikasi. Oleh karena itu, keluaran yang valid harus dapat diparsing dengan benar sekaligus memenuhi seluruh ketentuan yang telah dirancang.

Pengujian dilakukan menggunakan 50 query tugas akademik baru yang berasal dari berbagai mata kuliah dalam Program Studi Sistem Informasi. Setiap keluaran diperiksa untuk memastikan seluruh atribut yang diperlukan, yaitu *deadline_min_days*, *deadline_max_days*, *est_time_min_hours*, *est_time_max_hours*, *difficulty*, dan *reasoning*, berhasil diparsing dan ditampilkan pada dashboard. Selain itu, setiap keluaran juga diverifikasi agar memenuhi seluruh aturan pada prompt, seperti *deadline* minimum yang mempertimbangkan estimasi waktu pengerjaan secara wajar, selisih rentang *deadline* yang tidak melebihi 7 hari, serta selisih rentang estimasi waktu yang tidak melebihi 40% dari estimasi waktu minimum.

Berdasarkan hasil pengujian, seluruh keluaran berhasil memenuhi seluruh kriteria tersebut tanpa menghasilkan kesalahan parsing maupun pelanggaran terhadap aturan yang telah ditetapkan. Dengan demikian, nilai Parsing Success Rate (PSR) yang diperoleh sistem adalah sebagai berikut.

$$PSR = \frac{\text{jumlah keluaran yang memenuhi seluruh kriteria PSR}}{\text{total query uji}} = \frac{50}{50} = 100\%$$

Hasil tersebut menunjukkan bahwa sistem mampu menghasilkan keluaran yang konsisten, baik dari sisi struktur JSON maupun kepatuhan terhadap aturan yang diterapkan dalam prompt. Seluruh informasi estimasi, meliputi rentang *deadline*, rentang estimasi waktu pengerjaan, tingkat kesulitan, *reasoning*, serta referensi historis yang digunakan, berhasil ditampilkan pada dashboard aplikasi tanpa memerlukan intervensi manual.

BAB IV KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan hasil implementasi dan evaluasi yang telah dilakukan pada penelitian mengenai sistem estimasi beban tugas mahasiswa berbasis Retrieval-Augmented Generation (RAG), diperoleh beberapa kesimpulan sebagai berikut.

1. Sistem estimasi beban tugas mahasiswa berbasis Retrieval-Augmented Generation (RAG) berhasil diimplementasikan dalam bentuk dashboard berbasis Streamlit yang dapat digunakan untuk melakukan estimasi beban tugas akademik secara otomatis. Sistem mengintegrasikan proses semantic retrieval menggunakan SentenceTransformer multilingual-e5-base dan FAISS dengan model bahasa LLaMa 3.1 8B. Dashboard yang dikembangkan mampu menerima input berupa teks maupun dokumen PDF/DOCX, menampilkan hasil estimasi rentang deadline, rentang lama pengerjaan, tingkat kesulitan, reasoning, serta referensi tugas historis yang digunakan dalam proses estimasi.
2. Kinerja sistem Retrieval-Augmented Generation (RAG) yang dikembangkan dalam menghasilkan estimasi beban tugas akademik menunjukkan hasil yang baik dan konsisten pada seluruh komponen evaluasi. Pada tahap retrieval, sistem mencapai Hit Rate@3 sebesar 100% dan Precision@3 sebesar 80%, yang menunjukkan bahwa sistem mampu selalu menemukan minimal satu referensi historis yang relevan dan sebagian besar hasil retrieval memiliki keterkaitan yang baik dengan query. Pada evaluasi manual judgement, yang mencakup tiga aspek yaitu kesesuaian deadline, realisme estimasi waktu pengerjaan, dan kesesuaian difficulty, sistem memperoleh rata-rata masing-masing sebesar 4.16, 3.94, dan 3.98, yang menunjukkan bahwa hasil estimasi dinilai realistis dan sesuai dengan persepsi mahasiswa. Selanjutnya, pada evaluasi LLM-as-a-Judge yang terdiri dari aspek pemanfaatan referensi historis (3.60), keterkaitan reasoning dengan hasil estimasi (4.10), serta kejelasan dan koherensi reasoning (4.50), sistem memperoleh rata-rata keseluruhan sebesar 4.07 yang menunjukkan bahwa reasoning yang dihasilkan cukup baik, jelas, dan konsisten dalam mendukung hasil estimasi, meskipun pemanfaatan referensi historis masih perlu ditingkatkan. Selain itu, pada aspek parsing, sistem mencapai Parsing Success Rate sebesar 100% yang menunjukkan seluruh keluaran berhasil diproses dalam format JSON yang valid tanpa kesalahan. Secara keseluruhan, hasil ini menunjukkan bahwa sistem RAG yang dikembangkan mampu menghasilkan estimasi beban tugas yang relevan, realistis, dapat dijelaskan, serta terstruktur dengan baik, dan membuktikan bahwa pendekatan RAG yang umumnya digunakan pada tugas berbasis kualitatif juga efektif diadaptasikan untuk estimasi kuantitatif.

4.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, beberapa saran untuk pengembangan penelitian selanjutnya adalah sebagai berikut.

1. Menambah jumlah dan keragaman data historis tugas akademik dengan memperhatikan distribusi aktivitas utama yang dilakukan dalam pengerjaan tugas agar sistem memiliki referensi yang lebih banyak sehingga dapat meningkatkan kualitas retrieval dan akurasi estimasi yang dihasilkan.
2. Mengembangkan metode retrieval yang tidak hanya mempertimbangkan kemiripan semantik, tetapi juga relevansi topik, misalnya dengan menerapkan hybrid retrieval atau mekanisme re-ranking untuk meningkatkan kualitas referensi yang diberikan kepada model.
3. Melakukan evaluasi dengan jumlah responden dan jumlah query yang lebih besar agar hasil penilaian manual judgement dapat menggambarkan persepsi pengguna secara lebih komprehensif.
4. Melakukan perbandingan dengan pendekatan lain, seperti model machine learning regresi atau Large Language Model tanpa retrieval, sehingga kontribusi penggunaan Retrieval-Augmented Generation dapat dianalisis secara lebih mendalam.

DAFTAR PUSTAKA

- Aeon, B., & Aguinis, H. (2017). It's about time: New perspectives and insights on time management. *Academy of Management Perspectives*, 31(4), 309–330. <https://doi.org/10.5465/amp.2016.0166>
- Aggarwal, C. C. (2018). Machine learning for text. *Machine Learning for Text*, 1–493. <https://doi.org/10.1007/978-3-319-73531-3>
- AIS, A., & ACM, A. (2010). IS 2010: Curriculum guidelines for undergraduate degree programs in information systems. In *Communications of the Association for Information Systems* (Vol. 26, Number 1). <https://doi.org/10.17705/1cais.02618>
- Baik, C., Naylor, R., & Arkoudis, S. (2015). The First Year Experience in Australian Universities: Findings from 1994 to 2009. In *Centre for the Study of Higher Education University of Melbourne*. Melbourne Centre for the Study of Higher Education. http://www.cshe.unimelb.edu.au/research/experience/docs/FYE_Report_1994_to_2009.pdf
- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- Bourgeois, D., Smith, J., Roch, S., Fowler, J., Smith, B., Mortati, J., & WAN, S. (2014). Information Systems for Business and Beyond. *Information Systems for Business and Beyond*, 78. <http://www.saylor.org/courses/bus206>
- Brown, T. B., Krueger, G., Mann, B., Askell, A., Herbert-voss, A., Winter, C., Ziegler, D. M., Radford, A., & Mccandlish, S. (2020). *NeurIPS-2020-language-models-are-few-shot-learners-Paper*. (NeurIPS).
- Clark, R. E., Feldon, D., VanMerriënbour, J., Yates, K. A., & Early, S. (2008). Cognitive Task Analysis. *The SAGE Encyclopedia of Educational Technology*. <https://doi.org/10.4135/9781483346397.n56>
- Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated Evaluation of Retrieval Augmented Generation. *EACL 2024 - 18th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of System Demonstrations*, 150–158. <https://doi.org/10.18653/v1/2024.eacl-demo.16>
- Few, S. (2006). *Information Dashboard Design*. O'Reilly Media.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. Prmu. www.deeplearningbook.org
- Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Lin, Z., Zhang, B., Ni, L., Gao, W., Wang, Y., & Guo, J. (2026). A survey on LLM-as-a-Judge. *The Innovation*, 1(1), 101253. <https://doi.org/10.1016/j.xinn.2025.101253>
- Holmes, W., Bialik, M., & Fadel, C. (2019). *Artificial Intelligence in Education*. CENTER for CURRICULUM REDESIGN. <http://bit.ly/AIED-BOOK%0A>
- Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *EACL 2021 - 16th Conference of the European Chapter of the Association for Computational Linguistics, Proceedings of the Conference*, 874–880. <https://doi.org/10.18653/v1/2021.eacl-main.74>
- Johnson, J., Douze, M., & Jegou, H. (2021). Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>

- Jordon, J., Szpruch, L., Houssiau, F., Bottarelli, M., Cherubin, G., Maple, C., Cohen, S. N., & Weller, A. (2022). *Synthetic Data -- what, why and how?* <http://arxiv.org/abs/2205.03257>
- Jørgensen, M., & Shepperd, M. (2007). A Systematic Review of Software Development Cost Estimation Studies. *IEEE Transactions on Software Engineering (TSE)*, 33(1), 33–53.
- Joshi, A., Kale, S., Chandel, S., & Pal, D. (2015). Likert Scale: Explored and Explained. *British Journal of Applied Science & Technology*, 7(4), 396–403. <https://doi.org/10.9734/bjast/2015/14975>
- Jurafsky, D., & Martin, J. H. (2026). *Speech and Language Processing* (Third Edit). Pearson.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., & Küttler, H. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. (NeurIPS).
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Computing Surveys*, 55(9), 1–46. <https://doi.org/10.1145/3560815>
- Maha, L., Rahman, T., & Roy, C. (2026). Agile Story-Point Estimation: Is RAG a Better Way to Go? *34th IEEE/ACM International Conference on Program Comprehension (ICPC '26)*, 1(1). <https://doi.org/10.1145/3794763.3798175>
- Manning, D. C., Raghavan, P., & Schütze, H. (2009). Introduction to Modern Information Retrieval (2nd edition). In *Library Review* (Vol. 53, Number 9). <https://doi.org/10.1108/00242530410565256>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *1st International Conference on Learning Representations, ICLR 2013 - Workshop Track Proceedings*, 1–12.
- Misra, R., & Castillo, L. G. (2004). *Academic Stress Among College Students: Comparison of American and International Students*. 11(2), 132–148. <https://doi.org/10.1037/1072-5245.11.2.132>
- Misra, R., & McKean, M. (2000). College Student's Stress, and its Relationship to Their Anxiety, Time Management and Leisure Satisfaction. *American Journal of Health Studies*, 16, 11. <http://www.biomedsearch.com/article/College-students-academic-stress-its/65640245.html>
- Nikolenko, S. I. (2019). Synthetic data for deep learning. *Springer Optimization and Its Applications*, 174, 1–54. https://doi.org/10.1007/978-3-030-75178-4_1
- Ning, K., Pan, Z., Liu, Y., Jiang, Y., Zhang, J. Y., Rasul, K., Schneider, A., Ma, L., Nevmyvaka, Y., & Song, D. (2025). TS-RAG: Retrieval-Augmented Generation based Time Series Foundation Models are Stronger Zero-Shot Forecaster. (NeurIPS), 1–23. <http://arxiv.org/abs/2503.07649>
- Pascoe, M. C., Hetrick, S. E., & Parker, A. G. (2020). The impact of stress on students in secondary school and higher education. *International Journal of Adolescence and Youth*, 25(1), 104–112. <https://doi.org/10.1080/02673843.2019.1596823>
- Rich, E., & Knight, K. (2009). *Artificial Intelligence* (3rd ed.). Tata McGraw-Hill.
- Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach, Global Edition, 4ed* (Fourth Edit). Pearson.
- Satzinger, J. W., Jackson, R. B., & Burd, S. D. (2010). *Systems Analysis and Design in A Changing World Fifth Edition* (Fifth Edit). Course Technology Cengage Learning.
- Shorten, C., Pierse, C., Smith, T. B., Cardenas, E., Sharma, A., Trengrove, J., & van

- Luijt, B. (2024). *StructuredRAG: JSON Response Formatting with Large Language Models*. <http://arxiv.org/abs/2408.11061>
- Stair, R., & Reynolds, G. W. (2010). *Principles of Information Systems* (Ninth Edit). Course Technology Cengage Learning.
- Syafarina, G. A., & Zaenuddin, Z. (2023). Implementasi Framework Streamlit Sebagai Prediksi Harga Jual Rumah Dengan Linear Regresi. *Metik Jurnal*, 7 Nomor 2, 121–125. <https://doi.org/10.47002/metik.v7i2.680>
- Tang, X., Zong, Y., Zhao, Y., Cohan, A., & Gerstein, M. (2024). Are LLMs Good at Structured Outputs? A Benchmark for Evaluating Structured Output Capabilities in LLMs. *Information Processing and Management*, 61(6), 103809. <https://doi.org/10.1016/j.ipm.2024.103809>
- Tenckhoff, S., Koddenbrock, M., & Rodner, E. (2026). *LLMStructBench: Benchmarking Large Language Model Structured Data Extraction*. 1–14. <http://arxiv.org/abs/2602.14743>
- Tire, K., Taga, E. O., Ildiz, M. E., & Oymak, S. (2026). Retrieval Augmented Time Series Forecasting. *Proceedings of Machine Learning Research*, 267, 21774–21797.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 2017-Decem(Nips), 5999–6009. <https://doi.org/10.1201/9781003561460-19>
- Velcu-laitinen, O., & Yigitbasioglu, O. M. (2012). The Use of Dashboards in Performance Management : Evidence from Sales Managers. *The International Journal of Digital Accounting Research*, 12(November 2011), 39–58. <https://doi.org/10.4192/1577-8517-v12>
- Wang, L., Yang, N., Huang, X., Yang, L., Majumder, R., & Wei, F. (2024). *Multilingual E5 Text Embeddings: A Technical Report*. <http://arxiv.org/abs/2402.05672>
- Yang, S., Wang, D., Zheng, H., & Jin, R. (2024). TimeRAG: Boosting LLM Time Series Forecasting via Retrieval-Augmented Generation. *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*. <https://doi.org/10.1109/ICASSP49660.2025.10889933>
- Yu, H., Gan, A., Zhang, K., Tong, S., Liu, Q., & Liu, Z. (2025). Evaluation of Retrieval-Augmented Generation: A Survey. *Communications in Computer and Information Science*, 2301, 102–120. https://doi.org/10.1007/978-981-96-1024-2_8
- Zawacki-Richter, O., Marín, V. I., Bond, M., & Gouverneur, F. (2019). Systematic review of research on artificial intelligence applications in higher education – where are the educators? *International Journal of Educational Technology in Higher Education*, 16(1). <https://doi.org/10.1186/s41239-019-0171-0>
- Zheng, L., Chiang, W. L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36(NeurlPS), 1–29.

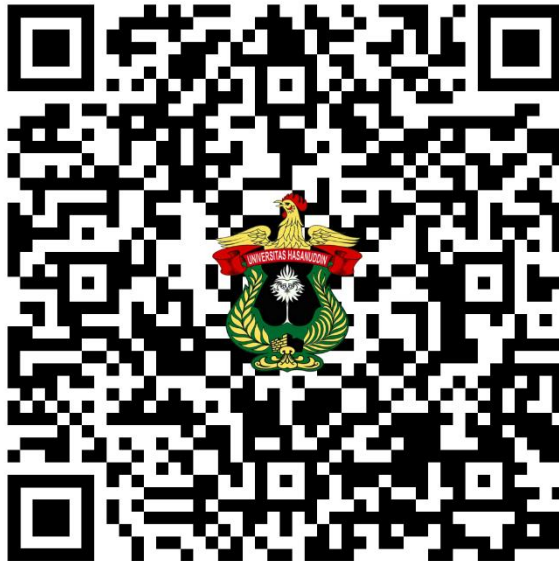
LAMPIRAN

Lampiran 1. Dataset Tugas Akademik



Lampiran 2. Preprocessing Dataset



Lampiran 3. Source Code Sistem**Lampiran 4. Evaluasi Retrieval**

Lampiran 5. Evaluasi LLM-as-a-Judge**Lampiran 6. Evaluasi Manual Judgement**